# MATPLOTLIB BASED WATERCRAFT HANDLER

Project Report Submitted to Bharathiar University in partial fulfillment of there requirements for the award of the degree of

**BACHELOR OF COMPUTER APPLICATIONS**

Submitted by


**ADHARSH K.V**

**(REG NO: 22BCA007)**

Under the Guidance of

**Mr. R. SATHISH MCA, M. Phil.,**

(Assistant Professor, Department of Computer Applications)

**DEPARTMENT OF COMPUTER APPLICATIONS**

**V.L.B. JANAKIAMMAL COLLEGE OF ARTS AND SCIENCE**

**(AUTONOMOUS)**

[AFFILIATED TO BHARATHIAR UNIVERSITY]

Re-Accredited by NAAC with 'B++' Grade Approved by UGC and AICTE

**KOVAIPUDUR, COIMBATORE– 641 042.**


**APRIL 2025**

# CERTIFICATE

**V.L.B. JANAKIAMMAL COLLEGE OF ARTS AND SCIENCE**

**AUTONOMOUS**

**KOVAIPUDUR, COIMBATORE– 641 042.**

**DEPARTMENT OF COMPUTER APPLICATIONS**

# CERTIFICATE

This is to certify that the project work entitled **"MATPLOTLIB BASED WATERCRAFT HANDLER"** is a Bonafide Record of **ADHARSH K.V** submitted to the Bharathiar University, in partial fulfillment of the requirements for the award of the Degree of **BACHELOR OF COMPUTER APPLICATIONS** during the year 2024 - 2025 in the Department of **Computer Applicatios** at **VLB Janakiammal College of Arts and Science**, Kovaipudur, Coimbatore – 641 042, under my supervision and guidance.

**PROJECT ADVISOR** **HEAD OF THE DEPARTMENT**

**Viva – Voce Examination held on_____**

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

# DECLERATION

# DECLARATION

I, **ADHARSH K.V (22BCA007)** hereby declare that the project entitled **"MATPLOTLIB BASED WATERCRAFT HANDLER"** submitted to the Bharathiar University in partial fulfillment of the award of the **BACHELOR OF COMPUTER APPLICATIONS** is an original project work done under the guidance of **Mr .R. SATHISH MCA, M. Phil.,** Assistant Professor, Department of Computer Applications, V.L.B. Janakiammal College of Arts and Science, Kovaipudur, Coimbatore.

**PLACE: COIMBATORE STUDENT SIGNATURE**

**DATE:** **ADHARSH KV**

# ACKNOWLEDGMENT

# ACKNOWLEDGEMENT

I would like to show my great appreciation to my parents for their continuous support during all these years. They were the first ones that introduced the amazing world to me and encouraged me to explore the wonderful nature.

I express my deep sense of gratitude to **THIRU. N. SURYA KUMAR,** Chairperson and Managing Trustee of VLBJ Trust and **Ms. JAISHREE SURYA KUMAR**, Secretary and CEO for providing the resource facilities and encouraging gesture for the completion of this project.

My genuine thanks to **Dr. D. KALIVANI M.SC., MCA, MPhil., PHD., PGDOM.,** Principal, VLB Janakiammal College of Arts and Science, Kovaipudur, Coimbatore- 641042, for providing all the facilities and permitting me to take up this project work.

My sincere thanks to **Mr. S. SHANTHGOHUL MCA., MPhil.,** Head of the Department Computer Applications, VLB Janakiammal College of Arts and Science, Kovaipudur, Coimbatore- 641042, for providing moral support towards completion of my project work.

My heartfelt thanks to my guide **Mr. R. SATHISH MCA, M. Phil.,** Assistant Professor, Department of Computer Applications, VLB Janakiammal College of Arts and Science, Kovaipudur, Coimbatore-641042, for his valuable advice and guidance. His timely suggestions and motivational talk enabled me to complete the dissertation on time; I express my sincere and profound gratitude to him.

Last, but not least, my Friends, also contributed much of their time and efforts to support me during my study. Without any one of them, the work would not have been possible. Thus, I also want to thank them for their love, support, and encouragement.

**ADHARSH K.V**

**22BCA007**

# INDEX

| Chapter No | CONTENTS | PageNo |
|---|---|---|
| 1 | **INTRODUCTION** | 1 |
| | 1.1Objective | |
| | 1.2Project overview | |
| | 1.3Module Description | 2 |
| 2 | **SYSTEM ANALYSIS** | 3 |
| | 2.1Existing System | |
| | 2.1.1Drawbacks of Existing System | |
| | 2.2Proposed System | |
| | 2.2.1Advantages of Proposed System | |
| 3 | **SYSTEM SPECIFICATION** | 4 |
| | 3.1Hardware Specification | |
| | 3.2Software Specification | |
| | 3.3Software Features | 5 |
| 4 | **SYSTEM DESIGN** | 9 |
| | 4.1File Design | |
| | 4.2Input Design | |
| | 4.3Database Design | |
| | 4.4 Output Design | 11 |
| 5 | **SYSTEMTESTING AND IMPLEMENTATION** | 12 |
| | 5.1SystemTesting | |
| | 5.2System Implementation | 14 |
| | 5.3System Maintenance | 15 |
| 6 | **CONCLUSION** | 16 |
| | **FUTURE ENHANCEMENT** | 17 |
| | **BIBLOGRAGHY** | 18 |
| | **APPENDIX** | 19 |
| | A. Data Flow Diagram | |
| | B. Entity Relationship Diagram | 22 |
| | C. Sample Screens | 23 |
| | D .Coding | 28 |

# ABSTRACT

We proposed matplotlib based water craft handler in the current web application to list the water craft details with high speed . It is the system design especially for watercraftbusiness and their functions. This website mainly provides boating services and also destination tourism. Here we can book a house boat and manage the same. User can access the application from anywhere and track their booking. Boating owners can get more business because of this. In this we focus more than one boating service under one platform concept. This project include 5 modules. Endorsement Module, Transaction Module, User Module, Admin Module. Boating Module. Endorsement Module is responsible to load different pages to the different users. Application has to show different pages on base of user role and different database transactions are managed by this module. Transaction Module is responsible for database transaction to handle different request from different users for booking purpose and tracking of the booking is controlled by this module. In Admin Module we can Login, Verify house boat owners, View tourists, View requests, Track payment. In User Module we can Registration, Login, Updating profile, Searching for boat details, Handle request, Make payment ,Track payment. In Water craftModule we can Registration, Handle profile ,Add watercraftdetails , View House boat ,details, Handle request, View Payment. The frontend of this project Django framework, python 3.8 , CSS, java script and the backend is MySQL server 5.6 which makes it easy for creating and generating code. Windows 10 is used as Operating System and hosting the web application in WSGI.

# 1. INTRODUCTION

## 1.1 Objective

The objective of the Matplotlib Based Watercraft Handler project is to develop a centralized and user-friendly web application that streamlines the booking and management of watercraft services, especially houseboats. This system aims to eliminate the drawbacks of manual processes by automating tasks such as user registration, boat listing, booking, and payment tracking through role-based modules for admins, users, and boat owners. By integrating a secure and normalized database, the system ensures data consistency, reduces redundancy, and allows quick information retrieval. Additionally, the use of Matplotlib enhances the application by providing visual insights into booking trends and service usage, helping both administrators and service providers make informed decisions. Ultimately, the platform is designed to improve user convenience, support business growth for boat owners, and offer a scalable, secure solution for managing multiple boating services under one digital ecosystem.

## 1.2 Project overview

This project proposes the development of a web application designed to efficiently manage watercraft details and facilitate related business functions, particularly for boating services and destination tourism. The core idea is to create a user-friendly platform where users can book houseboats, manage their bookings, and track their status from any location. Simultaneously, the system aims to empower boating owners by providing a unified platform to showcase their services and reach a wider audience, thereby increasing their business opportunities.

The application is structured into five key modules:

- Endorsement Module: Manages user access and displays appropriate content based on roles, handling database interactions.
- Transaction Module: Handles database transactions related to booking requests and their tracking.
- User Module: Enables user registration, login, profile management, boat searching, request handling, payment processing, and booking tracking.
- Admin Module: Provides administrative functionalities for login, verifying houseboat owners, viewing tourists and requests, and tracking payments.
- Boating Module: Allows watercraft owners to register, manage their profiles, add watercraft details, view houseboat information, handle requests, and view payment information.

The frontend of the application will be developed using the Django framework with Python 3.8, along with CSS and JavaScript for user interface and interactivity. The backend will utilize a MySQL 5.6 server for data management.

The project aims to overcome the inefficiencies and drawbacks of existing manual systems in the watercraft industry, such as reliance on paperwork, data redundancy, inconsistency, time-consuming processes, and lack of remote accessibility. By implementing a normalized database and a web-based interface, the proposed system promises to be more efficient, accurate, secure, user-friendly, and easily maintainable.

## 1.3 Module Discription

**Endorsement Module:**

- Responsibility: This module acts as the entry point and access control layer of the application. It determines which pages and functionalities are displayed to different users based on their roles (e.g., administrator, houseboat owner, general user).
- Functionality:
- Loads and renders different web pages dynamically based on the logged-in user's role.
- Manages user authentication and authorization to ensure appropriate access levels.
- Handles initial database connections and potentially sets up the user's session.
- Orchestrates the flow of control to other modules based on user actions and roles.

**Transaction Module:**

- Responsibility: This module is the core of the booking and tracking system. It manages all database interactions related to booking watercraft services.
- Functionality:
- Handles user requests for booking houseboats or other watercraft.
- Performs database transactions to record new bookings, update booking statuses, and cancel bookings.
- Manages the tracking of booking details, including dates, times, user information, and watercraft details.
- Potentially interacts with the Payment Module to process payments associated with bookings.
- Provides functionalities for users to view their booking history and current booking status.

**User Module**:

- Responsibility: This module manages the interactions and data related to general users (tourists, individuals looking to book watercraft).
- Functionality:
- Allows users to register new accounts with the system.
- Handles user login and logout processes.
- Enables users to update their personal profile information.
- Provides functionalities for searching and viewing available boat details based on various criteria (e.g., location, type, capacity).
- Allows users to submit booking requests for desired watercraft.
- Facilitates the process of making payments for bookings (likely interacting with the Transaction and potentially a separate Payment gateway integration).
- Enables users to track the status of their payments.

**Admin Module**:

- Responsibility: This module provides administrative functionalities for managing the overall system and the entities within it.
- Functionality:
- Allows administrators to log in to a dedicated administrative interface.
- Provides tools to verify and approve new houseboat owners or service providers before they can list their offerings.
- Enables administrators to view a list of registered tourists or general users.
- Allows administrators to view and manage all booking requests submitted by users.
- Provides functionalities for administrators to track the payment status of various bookings.

**Boating Module (Watercraft Module):**

- Responsibility: This module focuses on managing the information and operations related to the watercraft (specifically houseboats in your description) offered on the platform.
- Functionality:
- Allows registered watercraft owners to register their boats on the platform.
- Enables watercraft owners to manage their profile information and details.
- Provides a feature for watercraft owners to add detailed information about their boats, including specifications, availability, pricing, and images.
- Allows users (potentially both general users and administrators) to view the details of listed houseboats and other watercraft.
- Enables watercraft owners to handle booking requests received for their boats.
- Provides a way for watercraft owners to view payment information related to their bookings.

# 2. SYSTEM ANALYSIS

## 2.1Existing System

Existing System is handled manually. Manual system involves paper work in the form of writing exams, valuation, result publish etc. Maintaining critical information in the files and manuals is full of risk and a tedious process. Time loss and high maintenance cost are other disadvantages

### 2.1. 1Drawbacks of Existing System:

- **Manual check :** Checking the result of the test is tough.
- **Manual Question paper creation :**manual question paper creation is tough.
- **Redundancy:** Manual records tend to contain data which are redundant.
- **Inconsistency:** Because of redundant data, there is no consistency in the data.
- **Time Consuming:** Maintaining each details process and takes more time to retrieve back.
- **Tedious:** number of students data base goes through many numbers of updates each day which is hard to maintain in registers.

## 2.2 Proposed system

This VANCHIPURA overcomes all the above listed drawbacks.. All the students information, tests, test results are maintained in normalized database instead of manual records. This feature helps in maintaining database which is consistent, not redundant and easily maintainable. This system helps to establish exam system systematically

## 2.2.1 Advantages of Proposed System:

- A lot of manual work which is time consuming and tedious can be reduced.
- Eliminates wrong entries and hence give accurate report.
- Reduce paper work and extra cost.
- Data redundancy can be avoided.
- Data security.
- Information retrieval is faster.

# 3. SYSTEM SPECIFICATION

## 3.1Hardware Specification

Following are the hardware minimum required for the proposed system

| | | |
|---|---|---|
| Processor | : | Quad Core |
| Hard Disk | : | 120 GB |
| RAM | : | 4 GB |
| Monitor | : | Lenovo 15 inches |
| Mouse | : | Genius Scroll Mouse |
| Keyboard | : | 107 keys |

## 3.2Software Specifications

Following are the software minimum required for the proposed system

- Front End : PYTHON
- Back End : MYSQL
- Web Technologies : Html, Java Script, CSS

## 3.3 Software Features

**FRONT END**

Python is an interpreter, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding; make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms, and can be freely distributed.

Often, programmers fall in love with Python because of the increased productivity it provides. Since there is no compilation step, the edit-test-debug cycle is incredibly fast. Debugging Python programs is easy: a bug or bad input will never cause a segmentation fault. Instead, when the interpreter discovers an error, it raises an exception. When the program doesn't catch the exception, the interpreter prints a stack trace. A source level debugger allows inspection of local and global variables, evaluation of arbitrary expressions, setting breakpoints, stepping through the code a line at a time, and so on. The debugger is written in Python itself, testifying to Python's introspective power. On the other hand, often the quickest way to debug a program is to add a few print statements to the source: the fast edit-test-debug cycle makes this simple approach very effective.

**DJANGO FRAMEWORK**

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It’s free and open source.

## ADVANTAGES

### Complete

Django follows the "Batteries included" philosophy and provides almost everything developers might want to do "out of the box". Because everything you need is part of the one "product", it all works seamlessly together, follows consistent design principles, and has extensive and up-to-date documentation.

### Versatile

Django can be (and has been) used to build almost any type of website — from content management systems and wikis, through to social networks and news sites. It can work with any client-side framework, and can deliver content in almost any format (including HTML, RSS feeds, JSON, XML, etc). The site you are currently reading is built with Django!

### Secure

Django helps developers avoid many common security mistakes by providing a framework that has been engineered to "do the right things" to protect the website automatically. For example, Django provides a secure way to manage user accounts and passwords, avoiding common mistakes like putting session information in cookies where it is vulnerable (instead cookies just contain a key, and the actual data is stored in the database) or. directly storing password rather than a password hash. A password hash is a fixed-length value created by sending the password through a cryptographic hash function. Django can check if an entered password is correct by running it through the hash function and comparing the output to the stored hash value. However due to the "one-way" nature of the function, even if a stored hash value is compromised it is hard for an attacker to work out the original password. Django enables protection against many vulnerabilities by default, including SQL injection, cross-site scripting, cross-site request forgery and clickjacking (see Website security for more details of such attacks).

**Scalable**

Django uses a component-based “shared-nothing” architecture (each part of the architecture is independent of the others, and can hence be replaced or changed if needed). Having a clear separation between the different parts means that it can scale for increased traffic by adding hardware at any level: caching servers, database servers, or application servers. Some of the busiest sites have successfully scaled Django to meet their demands (e.g. Instagram and Disqus, to name just two).

**Maintainable**

Django code is written using design principles and patterns that encourage the creation of maintainable and reusable code. In particular, it makes use of the Don't Repeat Yourself (DRY) principle so there is no unnecessary duplication, reducing the amount of code. Django also promotesthe grouping of related functionality into reusable "applications" and, at a lower level, groups related code into modules (along the lines of the Model View Controller (MVC) pattern).

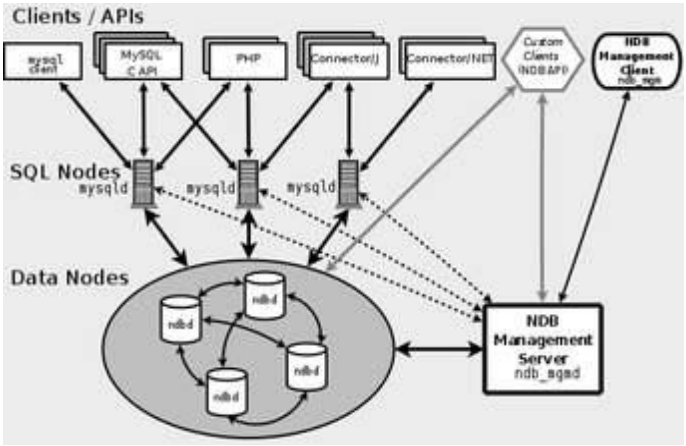
**Portable**

Django is written in Python, which runs on many platforms. That means that you are not tied to any particular server platform, and can run your applications on many flavours of Linux, Windows, and Mac OS X. Furthermore, Django is well-supported by many web hosting providers, who often provide specific infrastructure and documentation for hosting Django sites.

## BACK END

**SQLyog** provides you with powerful means to manage your MySQL databases, combining the features of MySQL Query Browser, Administrator, php My Admin and other MySQL Front Ends and MySQL GUI tools in a single intuitive interface. SQLyog is a fast, easy to use and compact graphical tool for managing your MySQL databases. SQLyog was developed for all who use MySQL as their preferred RDBMS. SQLyog provides you with powerful means to manage your MySQL databases. It features the simplicity of MySQL Front, with the power of EMS MySQL Admin. SQLyog provides detailed profile information for every SQL statement executed. SQLyog is a powerful MySQL manager that restores your previous session the way you left it. SQLyog allows easy access to frequently used SQL scripts. The script files can be stored as files from the SQLyog interface or link to an existing file anywhere where windows can access it - on a local drive or a shared network drive. The SQL scripts and the file links can be organized in folder and subfolders.

**Sql architecture:**



**FEATURES OF MYSQL:**

Relational Database System: Like almost all other database systems on the market, MySQL is a relational database system.

Client/Server Architecture: MySQL is a client/server system. There is a database server (MySQL) and arbitrarily many clients, which communicate with the server. The clients can run on the same computer as the server or on another computer.

SQL compatibility: MySQL supports as its database language -- as its name suggests – SQL (Structured Query Language). SQL is a standardized language for querying and updating data and for the administration of a database. There are several SQL dialects. MySQL adheres to the current SQL standard, although with significant restrictions and a large number of extensions.

Stored procedures: Stored procedures (SPs for short) are generally used to simplify certain steps, such as inserting or deleting a data record. For client programmers this has the advantage that they do not have to process the tables directly, but can rely on SPs. Like views, SPs help in the administration of large database projects. SPs can also increase efficiency. MySQL has supported SPs since version 5.0.User interface: There are a number of convenient user interfaces for administering a MySQL server.

# 4. SYSTEM DESIGN

## 4.1 File Design

File design is the process of producing a detailed data model of a database. This logical data model contains all the needed logical and physical design choices and physical storage parameters needed to generate a design in a Data Definition Language, which can then be used to create a database. A fully attributed data model contains detailed attributes for each entity.

The term database design can be used to describe many different parts of the design of an overall database system. Principally, and most correctly, it can be thought of as the logical design of the base data structures used to store the data. In the relational model these are the tables and views. In an Object database the entities and relationships map directly to object classes and named relationships. However, the term database design could also be used to apply to the overall process of designing, not just the base data structures, but also the forms and queries used as part of the overall database application within the Database Management System or DBMS.

## 4.2 Input Design

Input design is a part of overall system design, which requires careful attention. It is the process of converting user-originated inputs to a computer-based format. The major objective of the input design is to make data entry easy, logical and error free. In Visual Basic input to the system is entered through forms. A form is “any surface on which information is to be entered, the nature of which is determined by what is already on the surface”. If the data going into the system is incorrect, then processing and output will magnify these errors. So designer should ensure that form is acceptable and understandable by the user. This application has been developed in a user-friendly manner. The layout of the form is made in such a way that the user will not find any difficulty in going from one field to other by just pressing the tab. During the processing the cursor is placed in the position where the data must be entered. The user is also provided with an option of selecting an appropriate input from a list of values. Necessary dropdown list boxes and combo boxes are included for necessary fields so that the user need not remember all the data and can just select from it. Validation is made for each and every data entered. Help messages are also provided whenever the users enter a wrong data into a particular field. This makes the user to understand what is to be entered, moreover whenever an erroneous data is entered the error message is displayed and the user can move to the next field only after entering the correct data.

## 4.3 Database Design

The activity deals with the design of the database. A key is to determine how the access paths are to be implemented. A physical path is derived from a logical path. The general theme behind database is to handle information as a whole. A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make information access easy, quick, inexpensive and flexible for the user. Database design is the most critical part of the design phase. An elegantly designed, well-defined database is a strong foundation for the whole system. Files in a relational database are called as tables. Columns of tables represent data and rows represent the records in conventional technology.

**Normalization of Database**

Database Normalization is a technique of organizing the data in the database. Normalization is a systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics like Insertion, Update and Deletion Anomalies. It is a multi-step process that puts data into tabular form by removing duplicated data from the relation tables.

Normalization is used for mainly two purposes,

- Eliminating redundant (useless) data.
- Ensuring data dependencies make sense i.e. data is logically stored.

**Normalization rule**

Normalization rule are divided into following normal form.

- First Normal Form
- Second Normal Form
- Third Normal Form

**Database Table Design:**

Houseboat_tbl

| Slno | Attribute | type | Constraints | Description |
|---|---|---|---|---|
| 1 | no | MEDIUMINT(9) | PRIMARY KEY | no |
| 2 | firstname | VARCHAR(50) | NOTNULL | firstname |
| 3 | lastname | VARCHAR(50) | NOTNULL | lastname |

| | | | | |
|---|---|---|---|---|
| 4 | gender | VARCHAR(50) | NOTNULL | gender |
| 5 | dob | DATE | NOTNULL | dob |
| 6 | email | VARCHAR(50) | NOTNULL | email |
| 7 | phone | VARCHAR(50) | NOTNULL | phone |
| 8 | username | VARCHAR(20) | NOTNULL | username |
| 9 | password | VARCHAR(50) | NOTNULL | password |

Houseboaddetails_tbl

| Slno | Attribute | type | Constraints | Description |
|---|---|---|---|---|
| 1 | houseboatid | INT | PRIMARY KEY | houseboatid |
| 2 | boatname | VARCHAR(35) | NOTNULL | boatname |
| 3 | boatdetails | VARCHAR(500) | NOTNULL | boatdetails |
| 4 | boatimage | VARCHAR(30) | NOTNULL | boatimage |
| 5 | staydetails | VARCHAR(500) | NOTNULL | staydetails |
| 6 | pricedetails | VARCHAR(100) | NOTNULL | pricedetails |
| 7 | uploaderid | VARCHAR(20) | NOTNULL | uploaderid |
| 8 | uploadername | VARCHAR(35) | NOTNULL | uploadername |
| 9 | uploaddate | VARCHAR(35) | NOTNULL | uploaddate |

requesthandler_tbl

| Slno | Attribute | type | Constraints | Description |
|---|---|---|---|---|
| 1 | reqno | INT | PRIMARY KEY | reqno |
| 2 | touristno | VARCHAR(50) | NOTNULL | touristno |
| 3 | touristname | VARCHAR(50) | NOTNULL | touristname |
| 4 | houseboatno | VARCHAR(50) | NOTNULL | houseboatno |
| 5 | houseboatname | VARCHAR(50) | NOTNULL | houseboatname |
| 6 | staydetails | VARCHAR(50) | NOTNULL | staydetails |
| 7 | price | VARCHAR(50) | NOTNULL | price |
| 8 | date | VARCHAR(20) | NOTNULL | date |
| 9 | status | VARCHAR(50) | NOTNULL | status |
| 10 | uploaderid | VARCHAR(20) | NOTNULL | uploaderid |

tourist_tbl

| Slno | Attribute | type | Constraints | Description |
|---|---|---|---|---|
| 1 | no | MEDIUMINT(9) | PRIMARY KEY | no |
| 2 | firstname | VARCHAR(50) | NOTNULL | firstname |
| 3 | lastname | VARCHAR(50) | NOTNULL | lastname |
| 4 | gender | VARCHAR(50) | NOTNULL | gender |
| 5 | dob | DATE | NOTNULL | dob |

| | | | | |
|---|---|---|---|---|
| 6 | email | VARCHAR(50) | NOTNULL | email |
| 7 | phone | VARCHAR(20) | NOTNULL | phone |
| 8 | username | VARCHAR(20) | NOTNULL | username |
| 9 | password | VARCHAR(50) | NOTNULL | password |

**First Normal Form (1NF)**

- As per First Normal Form, no two Rows of data must contain repeating group of information i.e. each set of column must have a unique value, such that multiple columns cannot be used to fetch the same row. Each table should be organized into rows, and each row should have a primary key that distinguishes it as unique.
- The **Primary key** is usually a single column, but sometimes more than one column can be combined to create a single primary key. For example consider a table which is not in First normal form.

**Second Normal Form (2NF)**

As per the Second Normal Form there must not be any partial dependency of any column on primary key. It means that for a table that has concatenated primary key, each column in the table that is not part of the primary key must depend upon the entire concatenated key for its existence. If any column depends only on one part of the concatenated key, then the table fails **Second normal form**.

**Third Normal Form (3NF)**

**Third Normal form** applies that every non-prime attribute of table must be dependent on primary key, or we can say that, there should not be the case that a non-prime attribute is determined by another non-prime attribute. So this *transitive functional dependency* should be removed from the table and also the table must be in **Second Normal form**. For example, consider a table with following fields.

## 4.4 Output Design

One of the most important features of a system for users in the output it produces. Output design should improve the system's relationship with the user and help in decision-making. Considering the future use of output required, and depending on the nature, it is displayed on the monitor for immediate need of obtaining the hard copy.

The objective of output design is to define the controls and format of all printed documents and reports and of screens that will be produced by the system. Computer output is the most important and direct source of information to the user.

## Objectives Of Output Design

1. Design output to serve the intended purpose.
2. Deliver the appropriate quality of output.
3. Choose the right output method.
4. Provide output on time.

Output, generally refers to the results that are generated by the system. The output of the system is designed so as to include number of reports. Reports reflect the output design.

# 5.SYSTEM TESTING AND IMPLEMENTATION

## 5.1 System Testing

System testing includes code testing which examines the logic of the program. Each and every part of the program is checked or executed individually to find out the errors. Once the errors in the program are found out, they are debugged. If wrong data is entered, an error message is displayed on the screen so that the user can correct the data at that time itself.

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer based system. A series of testing are performed for the proposed system before the system is ready for the user acceptance test. A candidate system is subject to variety of tests – volume, stress, recovery, security and usability tests.

The steps in the system testing can be categorized as follows:

- Unit Testing
- Requirement Testing
- User Testing
- Validation Testing
- Integration Testing
- User Acceptance Testing

**Unit Testing**

Unit testing focuses on verification efforts on the smallest unit of software design i.e., the module. The unit testing is always white box oriented and the step can be conducted in parallel for modules. The module interface is tested to ensure that information properly flows in and out of the program unit under test. The ‘local data structures ‘ are examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm execution.” Boundary Conditions” are tested to ensure that the module operates properly at boundaries established to limit or restrict processing. All ‘independent paths ‘ through the control structures are exercised to ensure that all statement in a module have been executed at least once.

**Requirement Testing**

The main aim of this test plan is to see whether the outputs created and inputs were given according to the user requirements and specifications that have been established. This was done in the security department by having the developer as a secondary person and another employee who conducted the actual test. Some Suggestion was made while requirement testing was done that has been incorporated.

**User Testing**

During the testing the tester places the role of the individual who desires to penetrate the system. The tester may attempt to acquire passwords through external clerical means and may attack the system with the custom software design to break down any defenses that have been constructed. The tester may also overwhelm the system thereby denying service to other s and may purposely cause system errors to penetrate during recovery and may browse through insecure data, hoping to find key to system entry.

**Validation Testing**

At the end of user testing, software is completely assembled as a package, interfacing errors have now being uncovered and correcting test begins. Software testing and validation are achieved through a serious black box tests that demonstrate conformity with the requirement.

A plan outlines the classes of tests to be conducted and test procedure defines specific cases that will be used to demonstrate conformity with requirements. Both the plan and the procedure are designed to ensure that all functional requirements are achieved, documentation is correct and other requirements are met. After the validation test, one of the conditions exists.

The function or performance characteristics confirm to specification and are accepted. A deviation from the specification is uncovered and a deficiency list is created. The deviation or error discovered at this stage in a project can rarely be corrected prior to scheduled completion. It is necessary to negotiate with the customer to establish methods.

**Integration testing**

Integration testing is a systematic technique for constructing the program structure while conducting tests to uncover errors associated with interfacing. The objective is to take unit testing modules and build a program structure that has been dictated by design. There is often a tendency to attempt non incremental integration; that is to construct the program using "big-bang" approach. All modules are combined in advance. The entire program is tested as a whole. When a set of errors is encountered, correction is difficult because isolation of causes is complicated by the vast expanse of the entire program. Once these errors are corrected new ones appear and the process continues in a seemingly endless loop.Incremental integration is the antithesis of big-bang approach. The program is constructed and tested in small sequence, where errors are easier to isolate and correct; interfaces are more likely to be tested completely; and a systematic test approach may be applied.

**User acceptance testing**

Acceptance testing involves planning and execution of functional tests, performance test, and stress tests to verify that the implemented system satisfies its requirements. Acceptance tests are typically performed by quality assurance and/or customer organizations. Functional and performance tests are performed to determine the limitations of the system. Typically , acceptance typically, acceptance test will incorporate test cases developed during unit testing and integration testing. Additional test cases are added to achieve desired level of functional , performance, and stress testing of the entire system. Tools of Special importance during acceptance testing include a test Coverage analyzer, a timing analyzer, and a coding standards checker. A test coverage analyzer records the control paths followed for each test case. Timing analyzer reports the time spent in various regions of the source code and different test cases.

**White Box Testing**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

**Black Box Testing**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

## 5.2 System Implementation

**Implementation Planning:**

This section describes about the Implementation of the Safe application and the details of how to access this control from any application.

Implementation is the process of assuring that the information system is operational and then allowing users take over its operation for use and evaluation. Implementation includes the following activities.

- Obtaining and installing the system hardware.
- Installing the system and making it run on its intended hardware.
- Providing user access to the system.
- Creating and updating the database.
- Documenting the system for its users and for those who will be responsible for maintaining it in the future.
- Making arrangements to support the users as the system is used.
- Transferring ongoing responsibility for the system from its developers to the operations or maintenance part.
- Evaluating the operation and use of the system.

**Implementation Phase in this project:**

The new system of Electronic Land Information Management System has been implemented. The present system has been integrated with the already existing hardware. The database was put into the Microsoft SQL server. This was connected by JDBC. The database is accessible through Internet on any geographic location. Documentation is provided well in such a way that it is useful for users and maintainers.

**Maintenance:**

Maintenance is any work done to change the system after it is in operational. The term maintenance is used to describe activities that occur following the delivery of the product to the customer. The maintenance phase of the software life cycle is the time period in which a software product performs useful work.

Maintenance activities involve making enhancements to products, adapting products to new environments, correcting problems.

In this be retrieve the data from the database design by searching the database. So, for maintaining data our project has a backup facility so that there is an additional copy of data, which needs to be maintained.

More over this project would update the annual data on to a CD, which could be used for later reference

**Methods of implementation:**

The four basic methods of implementation are:

1. Parallel system method.
2. Direct cut over method.
3. Pilot system method.
4. Phase in method.

## 5.3 System Maintenance

System maintenance is widely accepted part of SDLC now a days. It stands for all the modifications and updations done after the delivery of software product. There are number of reasons, why modifications are required, some of them are briefly mentioned below:

- **Market Conditions** - Policies, which changes over the time, such as taxation and newly introduced constraints like, how to maintain bookkeeping, may trigger need for modification.
- **Client Requirements** - Over the time, customer may ask for new features or functions in the software.
- **Host Modifications** - If any of the hardware and/or platform (such as operating system) of the target host changes, software changes are needed to keep adaptability.
- **Organization Changes** - If there is any business level change at client end, such as reduction of organization strength, acquiring another company, organization venturing into new business, need to modify in the original software may arise.

### Types of maintenance

In a software lifetime, type of maintenance may vary based on its nature. It may be just a routine maintenance tasks as some bug discovered by some user or it may be a large event in itself based on maintenance size or nature. Following are some types of maintenance based on their characteristics:

- **Corrective Maintenance** - This includes modifications and updations done in order to correct or fix problems, which are either discovered by user or concluded by user error reports.
- **Adaptive Maintenance** - This includes modifications and updations applied to keep the software product up-to date and tuned to the ever changing world of technology and business environment.

- **Perfective Maintenance -** This includes modifications and updates done in order to keep the software usable over long period of time. It includes new features, new user requirements for refining the software and improve its reliability and performance.

# 6.CONCLUSION

The Matplotlib-Based Watercraft Handler offers a significant improvement over existing manual systems by providing a centralized, efficient, and user-friendly platform for managing watercraft services. The implementation of a normalized database ensures data consistency, eliminates redundancy, and simplifies maintenance. The modular design caters to various user roles, streamlining processes from booking and payment to administration and watercraft management. By leveraging web technologies like Django and MySQL, the system promises enhanced accessibility, security, and scalability. Ultimately, this digital solution aims to reduce manual workload, improve accuracy, accelerate information retrieval, the Matplotlib-Based Watercraft Handler offers a comprehensive solution to the challenges of managing watercraft services manually. Its digital approach promises significant advantages in terms of efficiency, accuracy, user experience, and business growth, laying a solid foundation for future development and innovation in the watercraft tourism sector.

# FUTURE ENHANCEMENT

The system has been designed and developed according to the current requirements of the user. At the same time the system is very flexible and extensible. Hence, future enhancements, if needed can be made without much difficulty. So new applications can be developed and it will be integrated with the existing one very easily.

The packages were designed in such a way that future modification can be done easily. The following future enhancements may be worthwhile to make the tool usable for a wide section or users.

- Additional Categorization on base of region.
- More payment options.

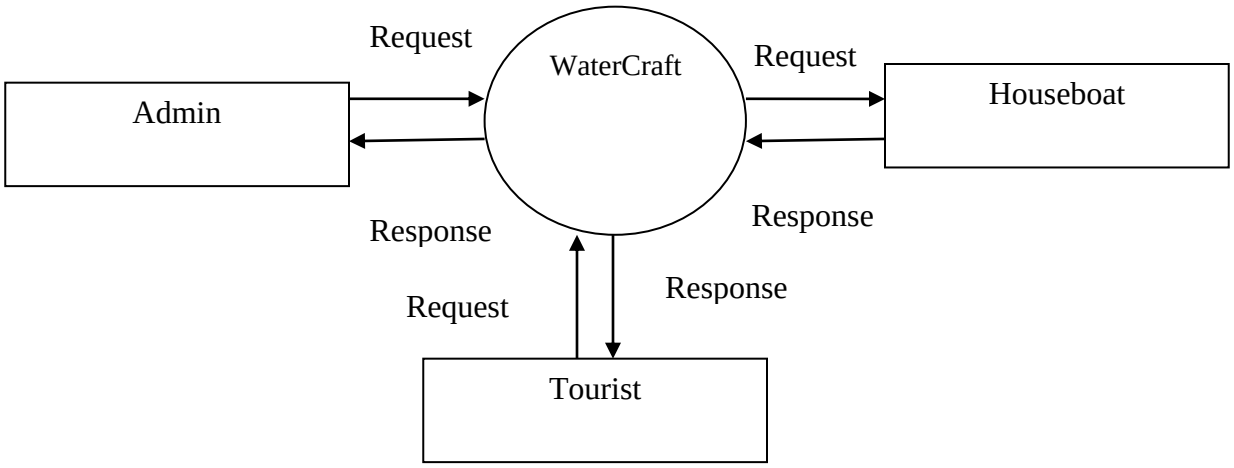
# BIBLOGRAGHY


1. Python Crash Course, 2nd EditionEric Matthes
2. Learning Python, 5th EditionMark Lutz
3. Automate the Boring Stuff with Python, 2nd Edition Al Sweigart
4. Python for Everybody: Exploring Data in Python 3 Charles Severance
5. Python (2nd Edition): Learn Python in One Day and Learn It Well. LCF Publishing, Jamie Chan
6. Coding in PythonTips and Tricks to Coding with Python Using the Principles and Theories of Python Programming
7. 2 books in 1: learn python programming for beginners and machine learningWillard D Sanders

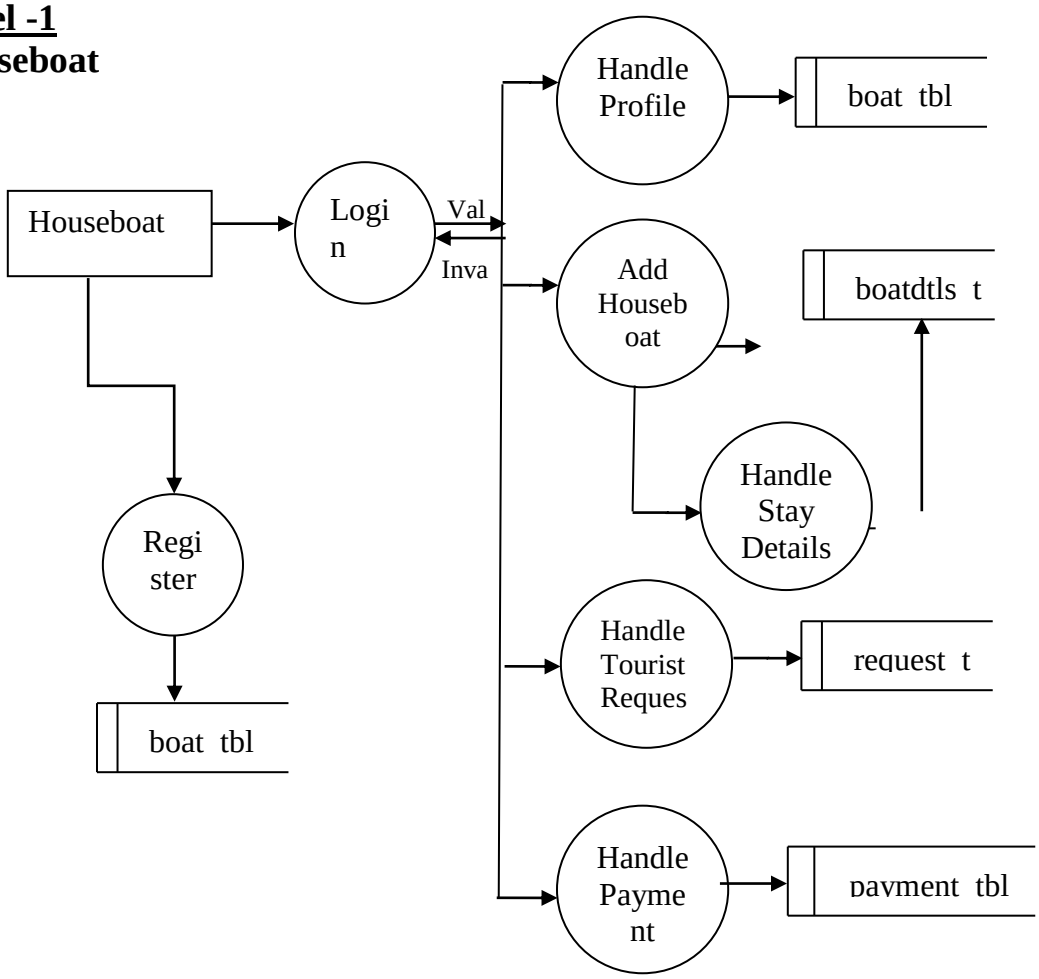
# APPENDIX

## A. Data Flow Diagram

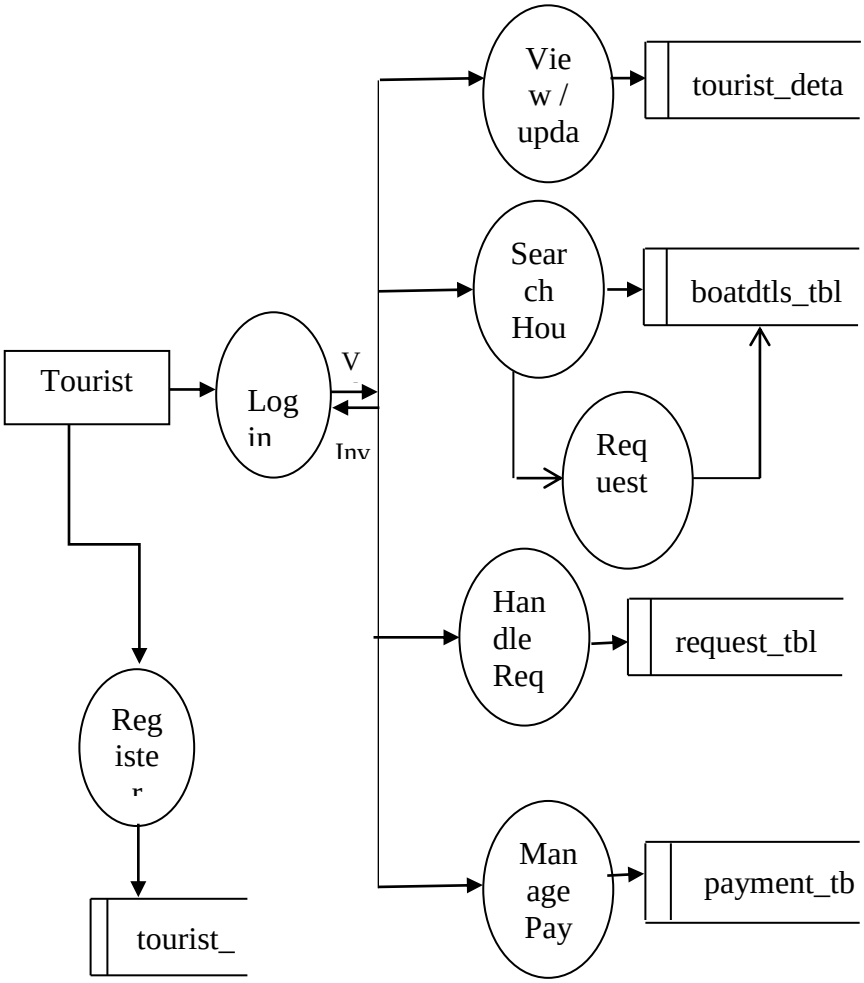
### Level 0



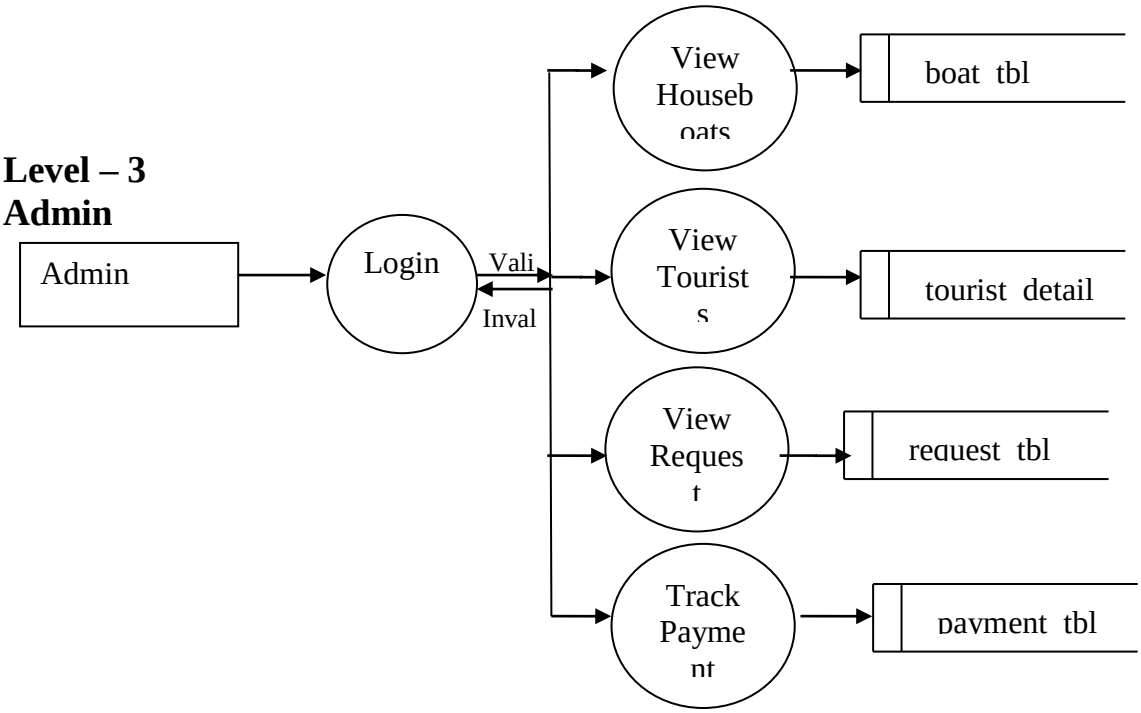

### Level -1
### Houseboat



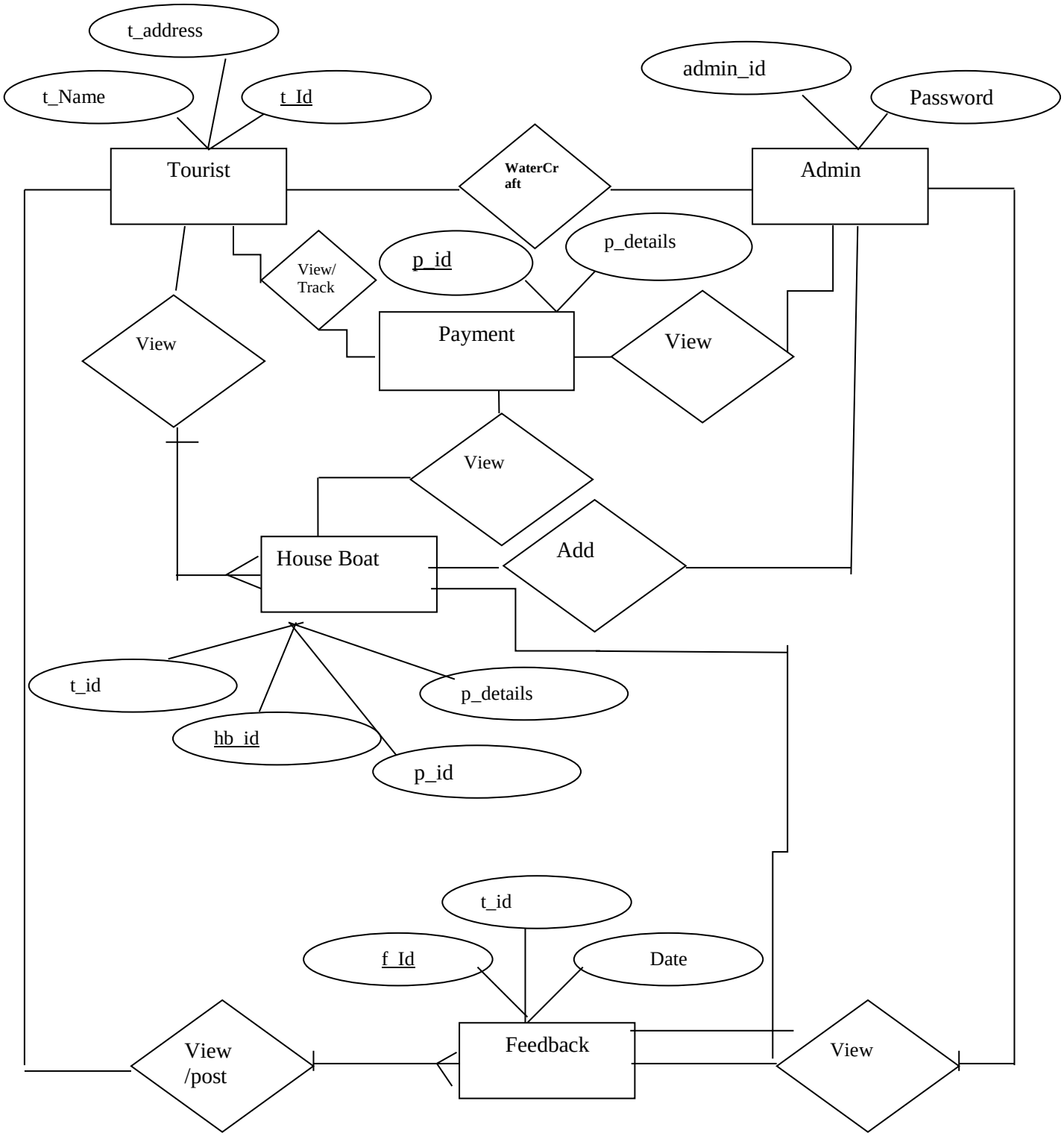
**Level -2**
**Tourist**



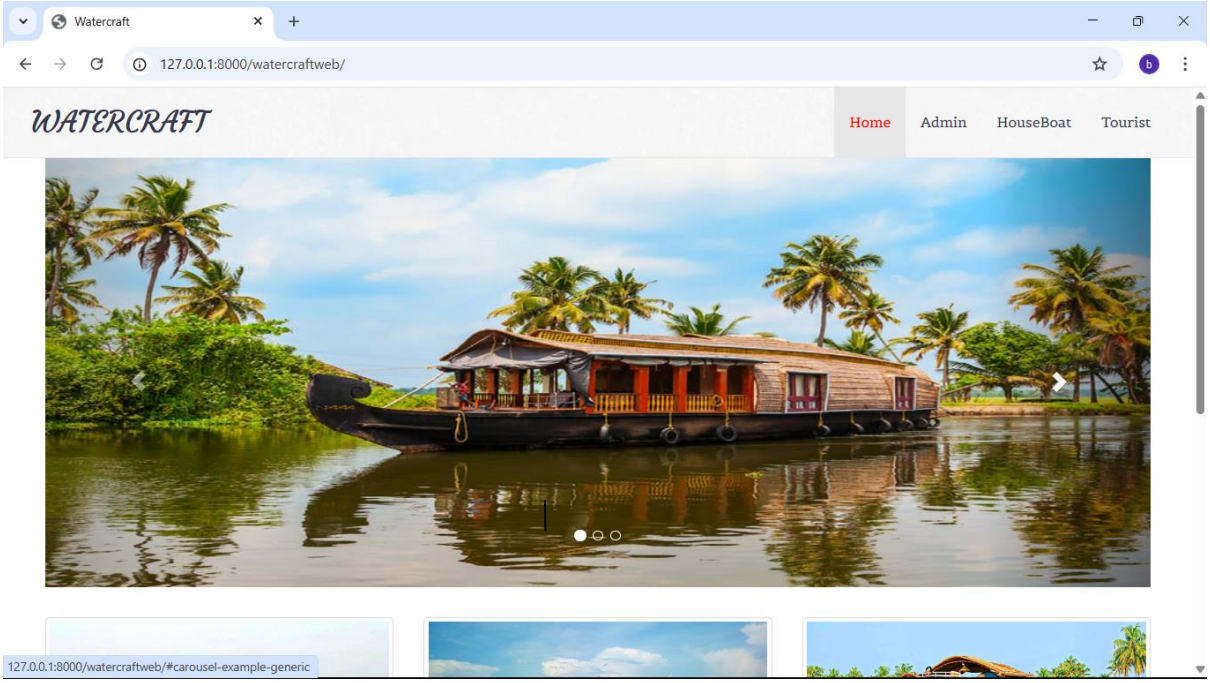

**Level – 3**
**Admin**

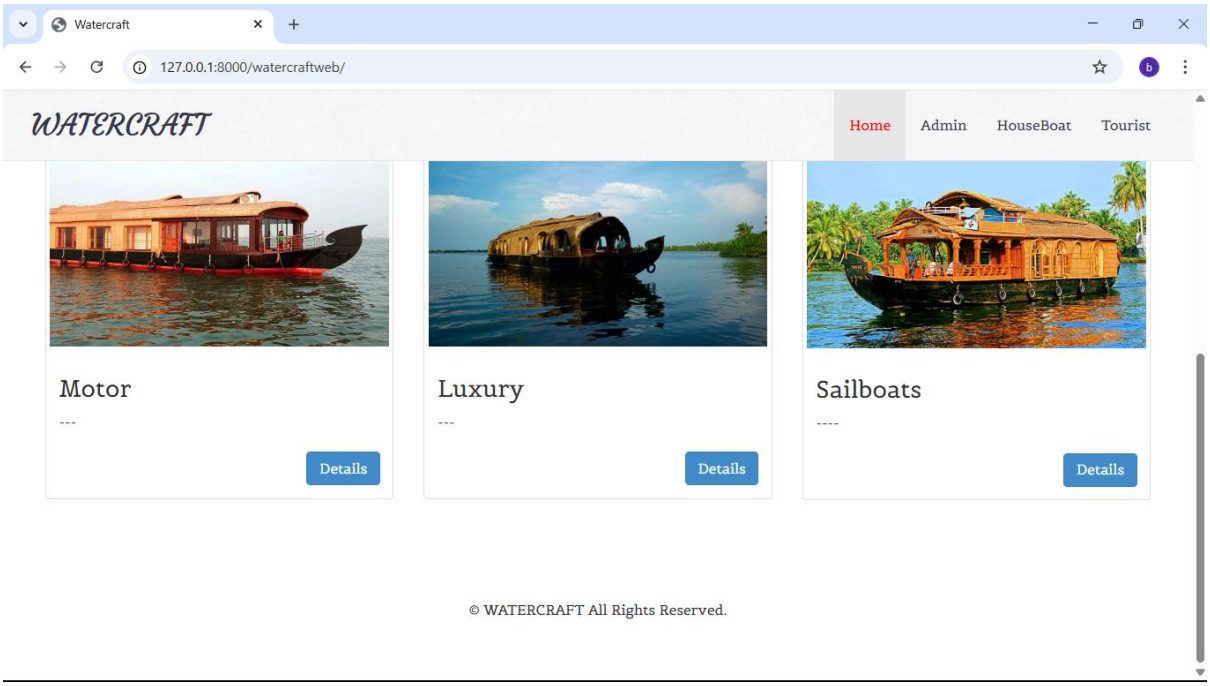


## B. Entity Relationship Diagram

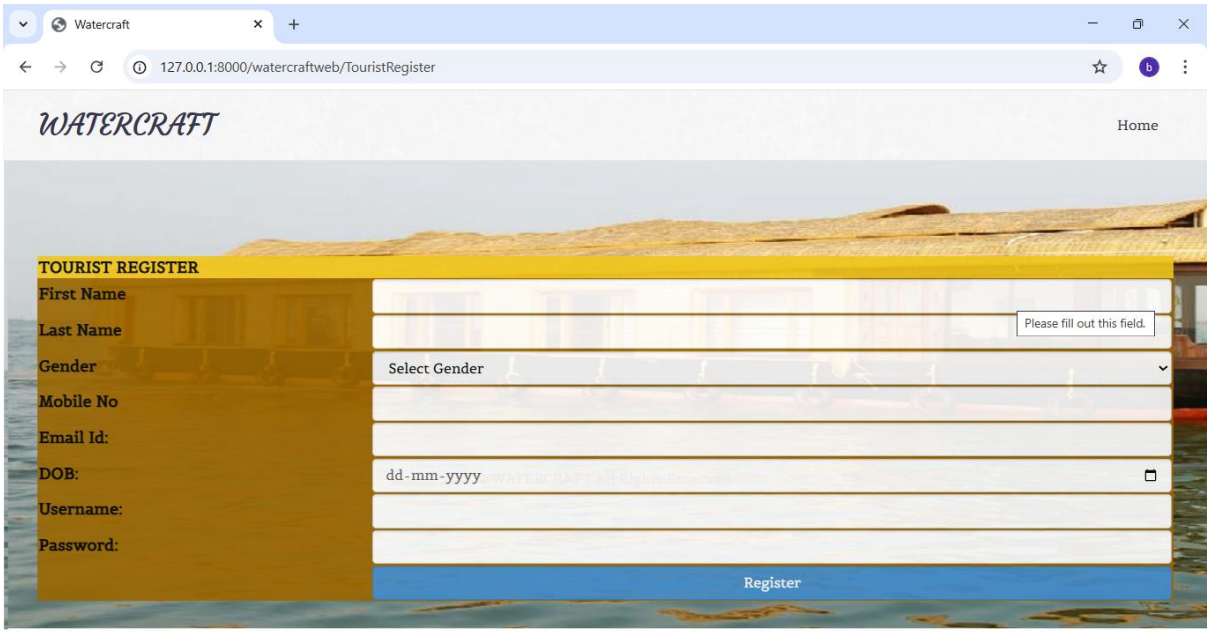


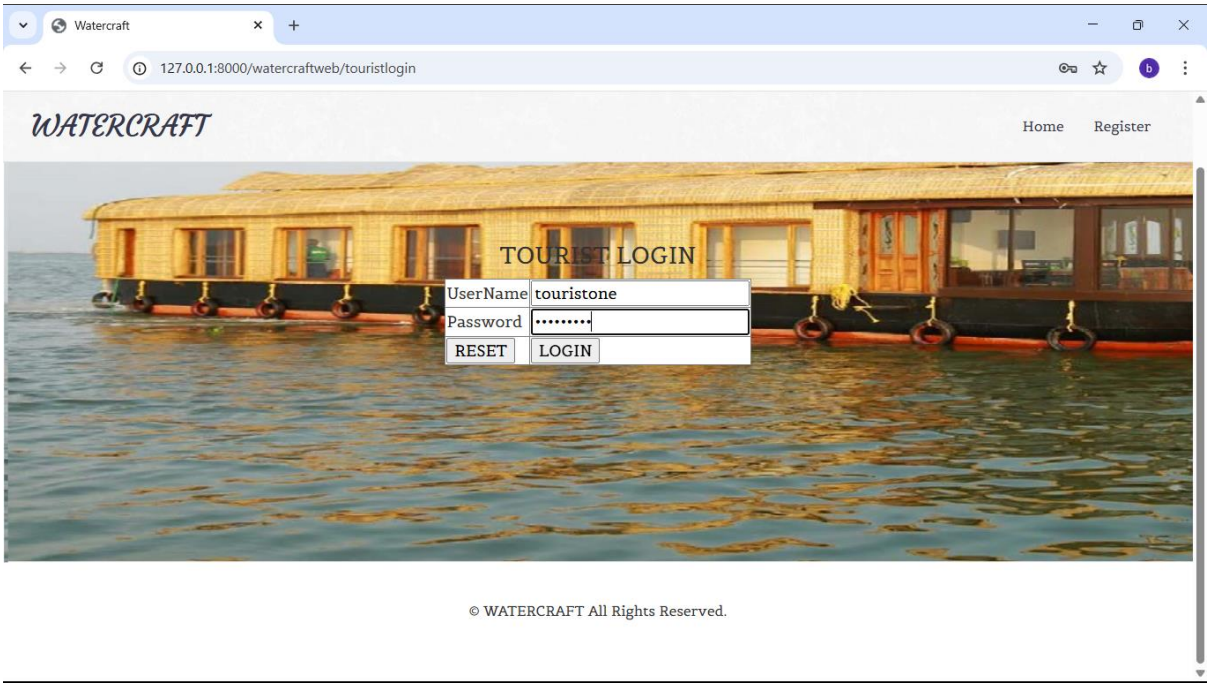
# C. Sample Screens

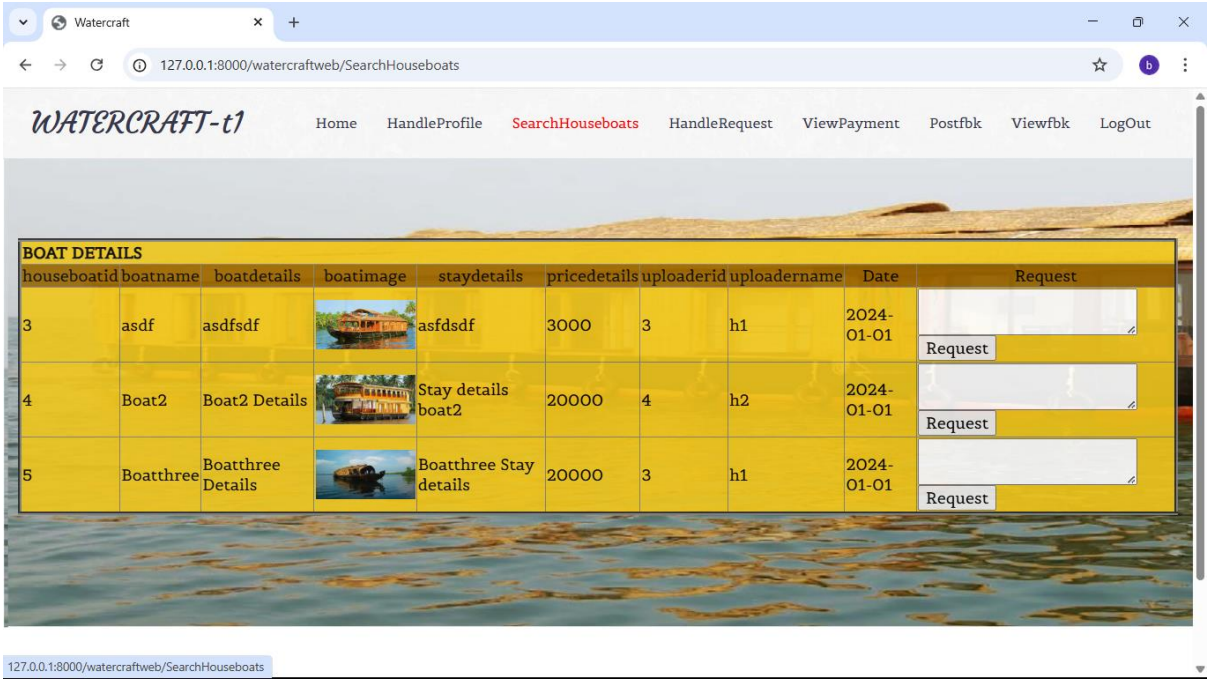


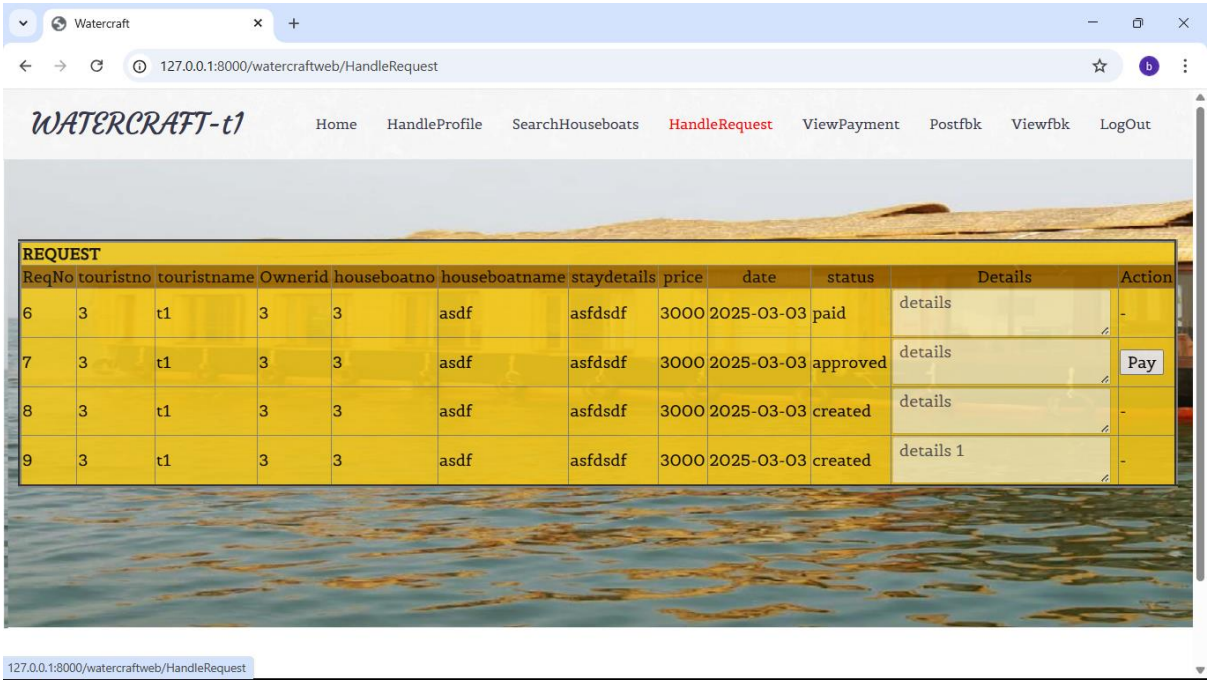



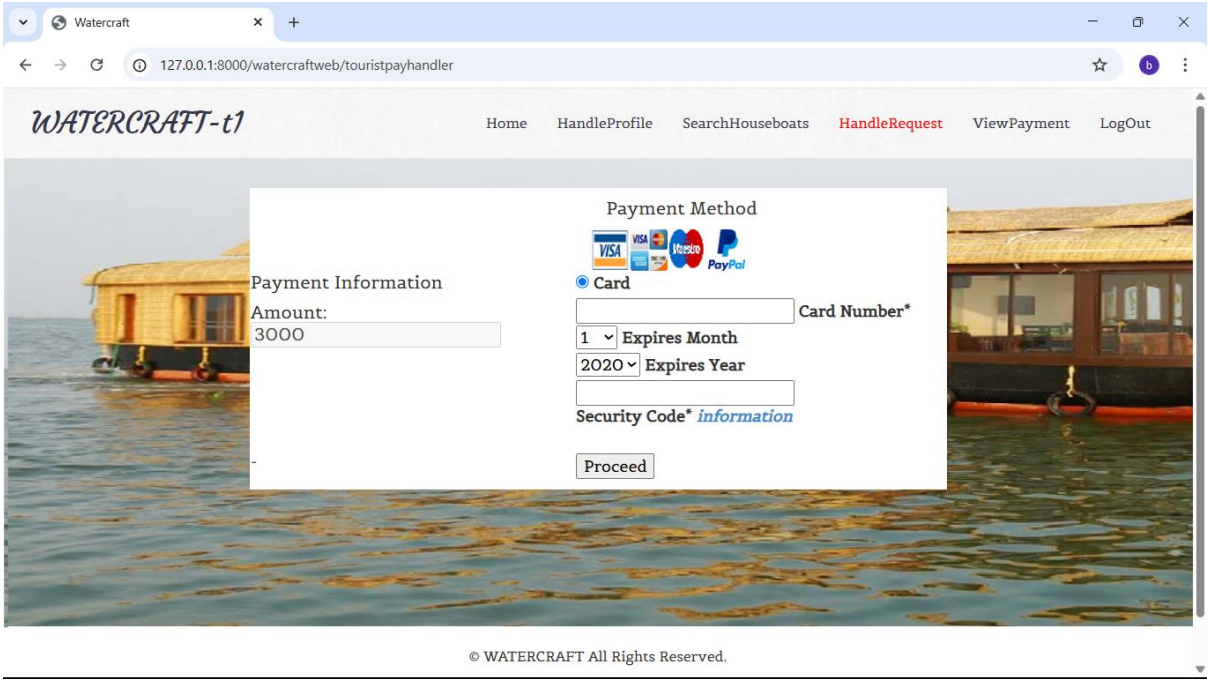
## Tourist

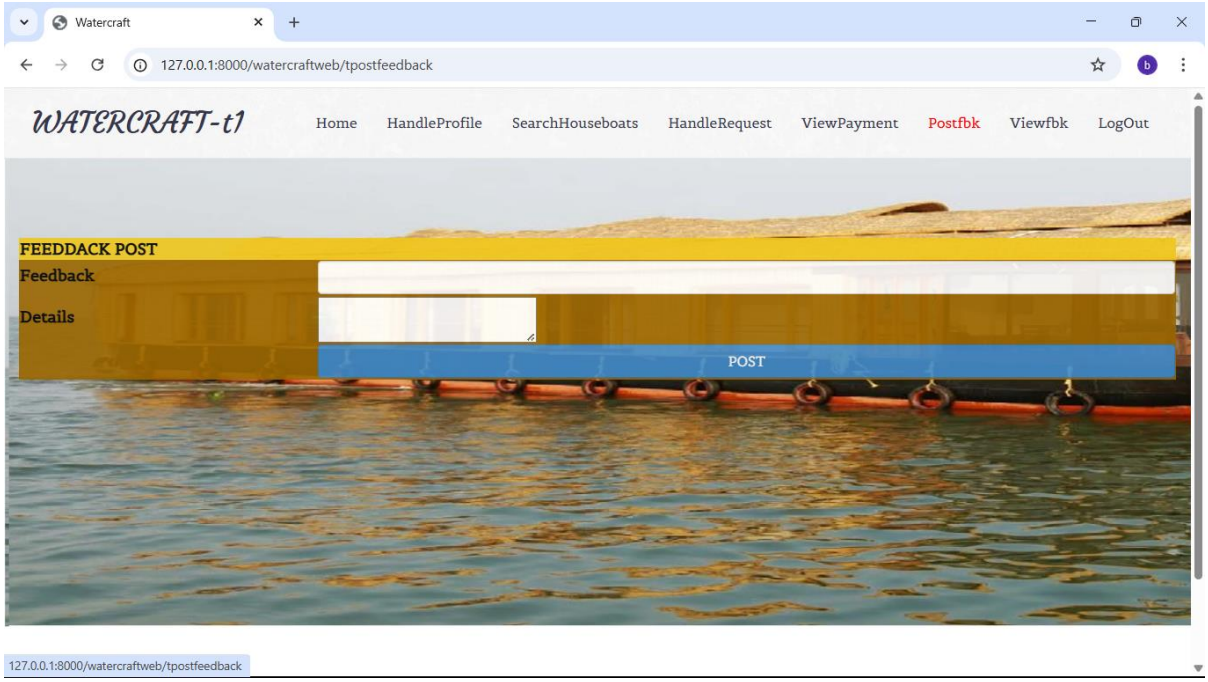

Watercraft
127.0.0.1:8000/watercraftweb/TouristRegister
WATERCRAFT
Home
TOURIST REGISTER
First Name
Last Name
Please fill out this field.
Gender
Select Gender
Mobile No
Email Id:
DOB:
dd-mm-yyyy
Username:
Password:
Register


Watercraft
127.0.0.1:8000/watercraftweb/touristlogin
WATERCRAFT
Home
Register
TOURIST LOGIN
UserName
touristone
Password
RESET
LOGIN
© WATERCRAFT All Rights Reserved.

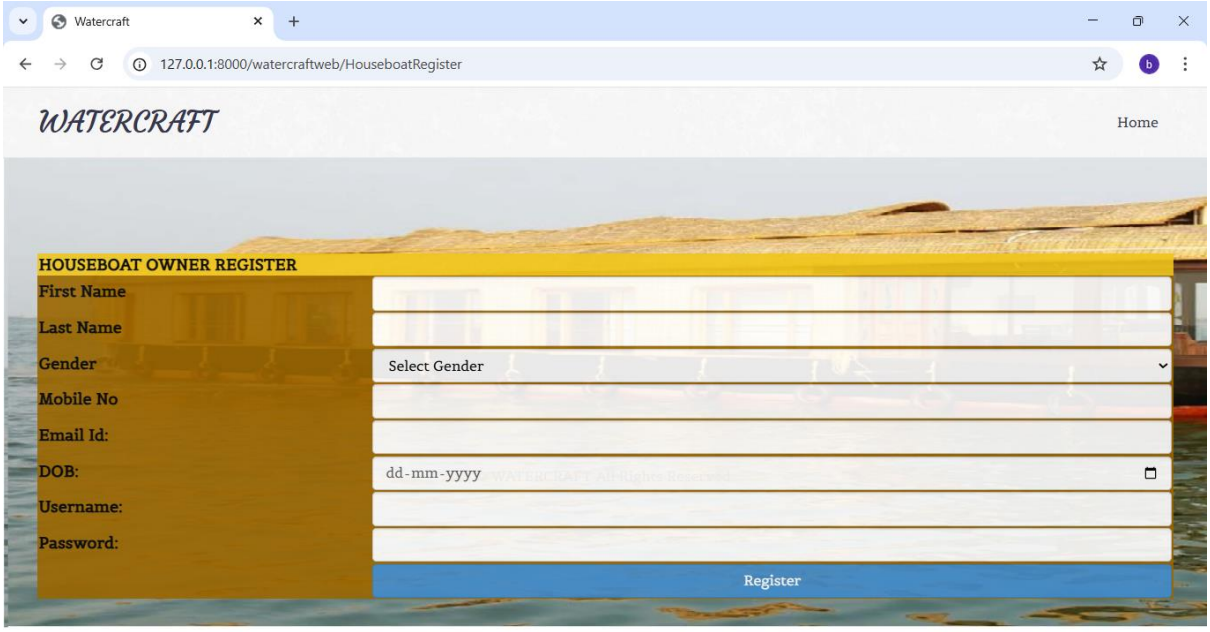

Watercraft
127.0.0.1:8000/watercraftweb/SearchHouseboats
WATERCRAFT-t1
Home
HandleProfile
SearchHouseboats
HandleRequest
ViewPayment
Postfbk
Viewfbk
LogOut
BOAT DETAILS
houseboatid boatname boatdetails boatimage staydetails pricedetails uploaderid uploadername Date Request
3 asdf asdfsdf asfdsdf 3000 3 h1 2024-01-01 Request
4 Boat2 Boat2 Details Stay details boat2 20000 4 h2 2024-01-01 Request
5 Boatthree Boatthree Details Boatthree Stay details 20000 3 h1 2024-01-01 Request
127.0.0.1:8000/watercraftweb/SearchHouseboats

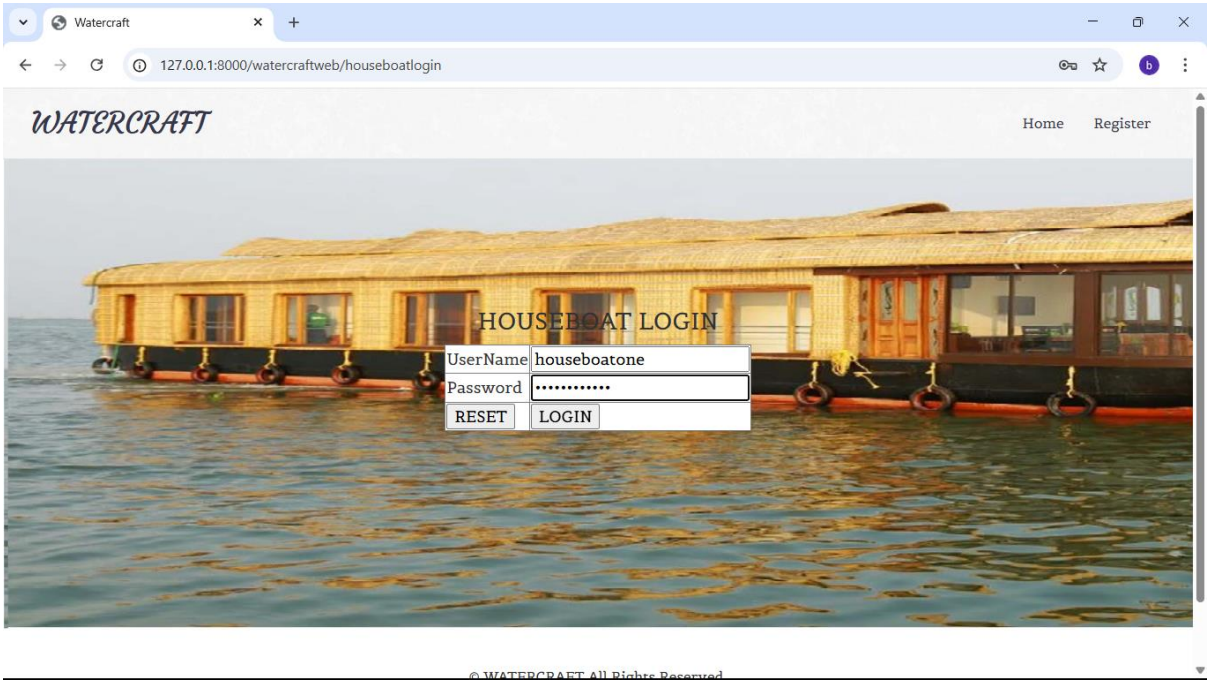

Watercraft
127.0.0.1:8000/watercraftweb/HandleRequest
WATERCRAFT-t1
Home
HandleProfile
SearchHouseboats
HandleRequest
ViewPayment
Postfbk
Viewfbk
LogOut
REQUEST
ReqNo touristno touristname Ownerid houseboatno houseboatname staydetails price date status Details Action
6 3 t1 3 3 asdf asfdsdf 3000 2025-03-03 paid details -
7 3 t1 3 3 asdf asfdsdf 3000 2025-03-03 approved details Pay
8 3 t1 3 3 asdf asfdsdf 3000 2025-03-03 created details -
9 3 t1 3 3 asdf asfdsdf 3000 2025-03-03 created details 1 -
127.0.0.1:8000/watercraftweb/HandleRequest

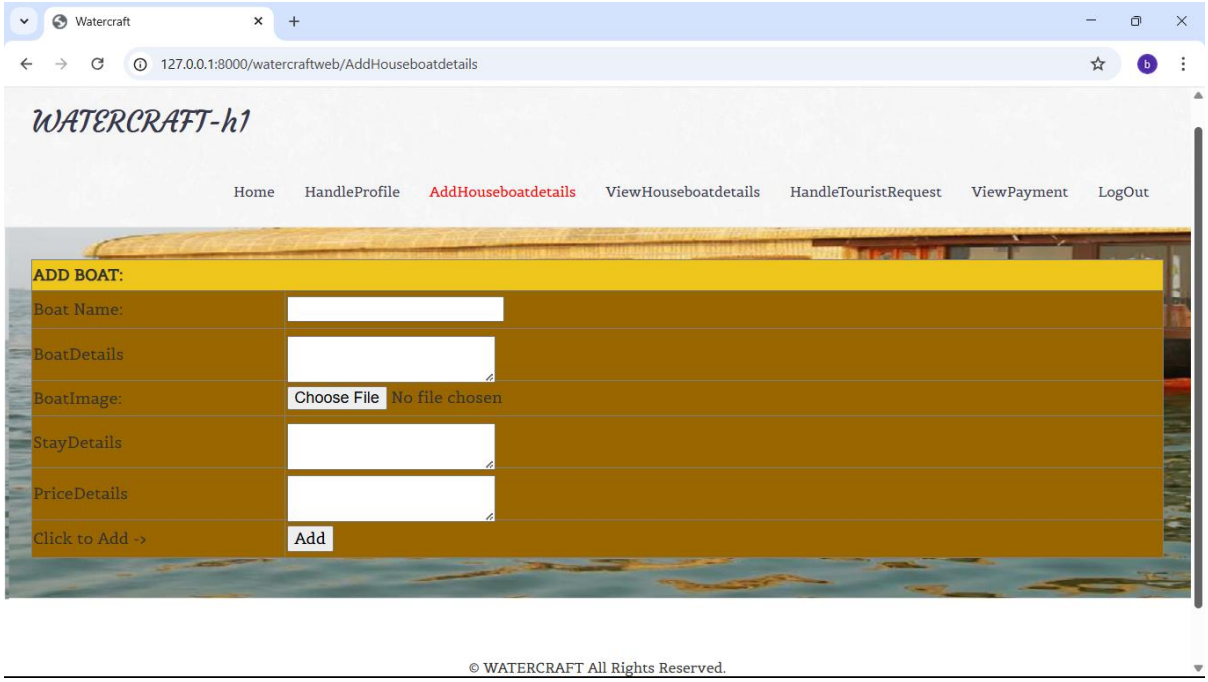

Watercraft
127.0.0.1:8000/watercraftweb/touristpayhandler
WATERCRAFT-t1
Home
HandleProfile
SearchHouseboats
HandleRequest
ViewPayment
LogOut
Payment Method
Payment Information
Amount:
3000
Card
Card Number*
1
Expires Month
2020
Expires Year
Security Code* information
Proceed
© WATERCRAFT All Rights Reserved.

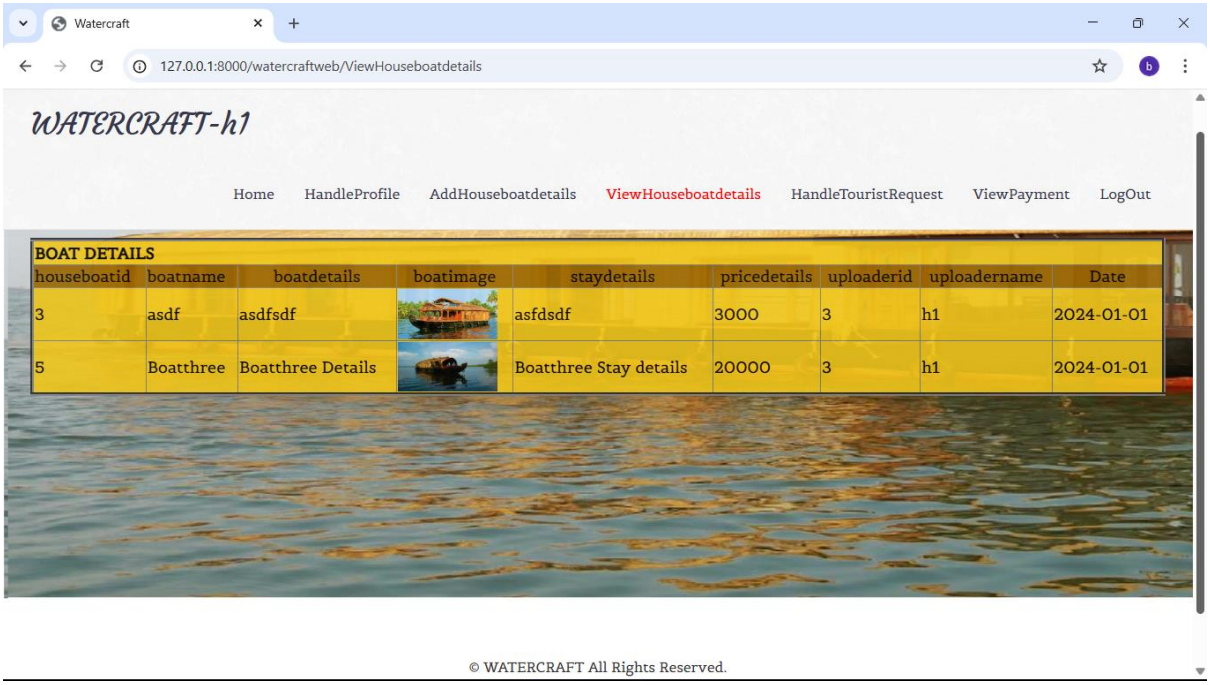

Watercraft
127.0.0.1:8000/watercraftweb/tpostfeedback
WATERCRAFT-t1
Home
HandleProfile
SearchHouseboats
HandleRequest
ViewPayment
Postfbk
Viewfbk
LogOut
FEEDDACK POST
Feedback
Details
POST
127.0.0.1:8000/watercraftweb/tpostfeedback

House Boat


Watercraft
127.0.0.1:8000/watercraftweb/HouseboatRegister
WATERCRAFT
Home
HOUSEBOAT OWNER REGISTER
First Name
Last Name
Gender
Select Gender
Mobile No
Email Id:
DOB:
dd-mm-yyyy
Username:
Password:
Register

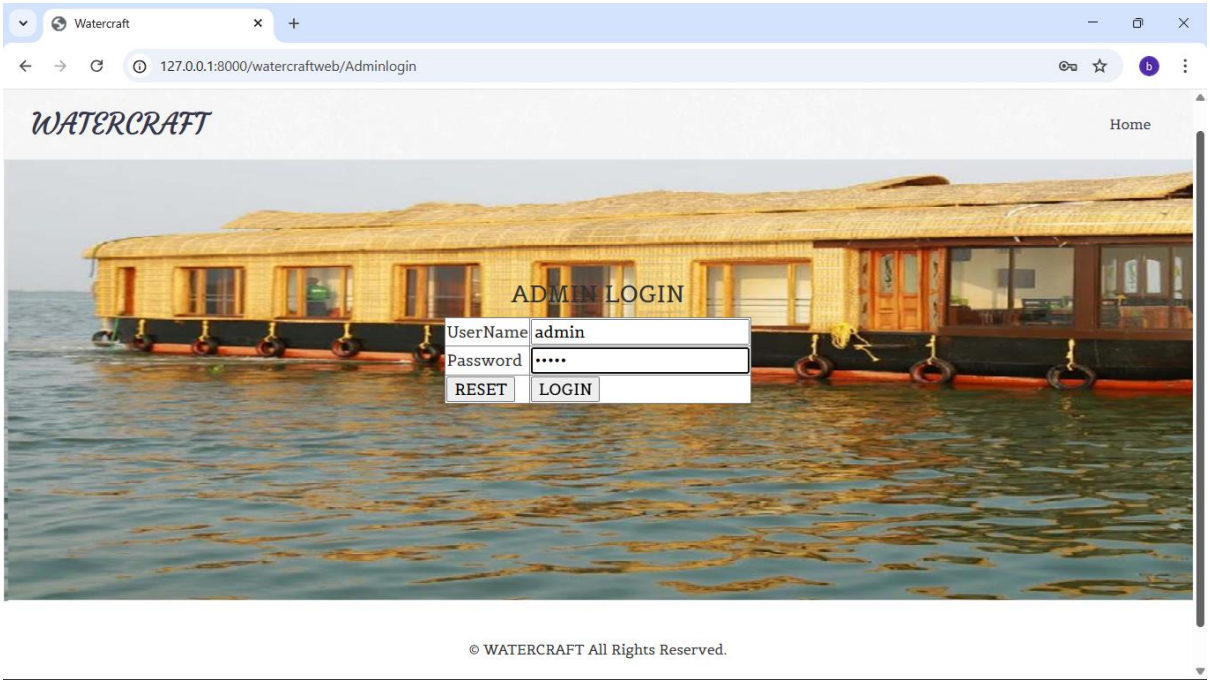

Watercraft
127.0.0.1:8000/watercraftweb/houseboatlogin
WATERCRAFT
Home
Register
HOUSEBOAT LOGIN
UserName
houseboatone
Password
RESET
LOGIN
© WATERCRAFT All Rights Reserved

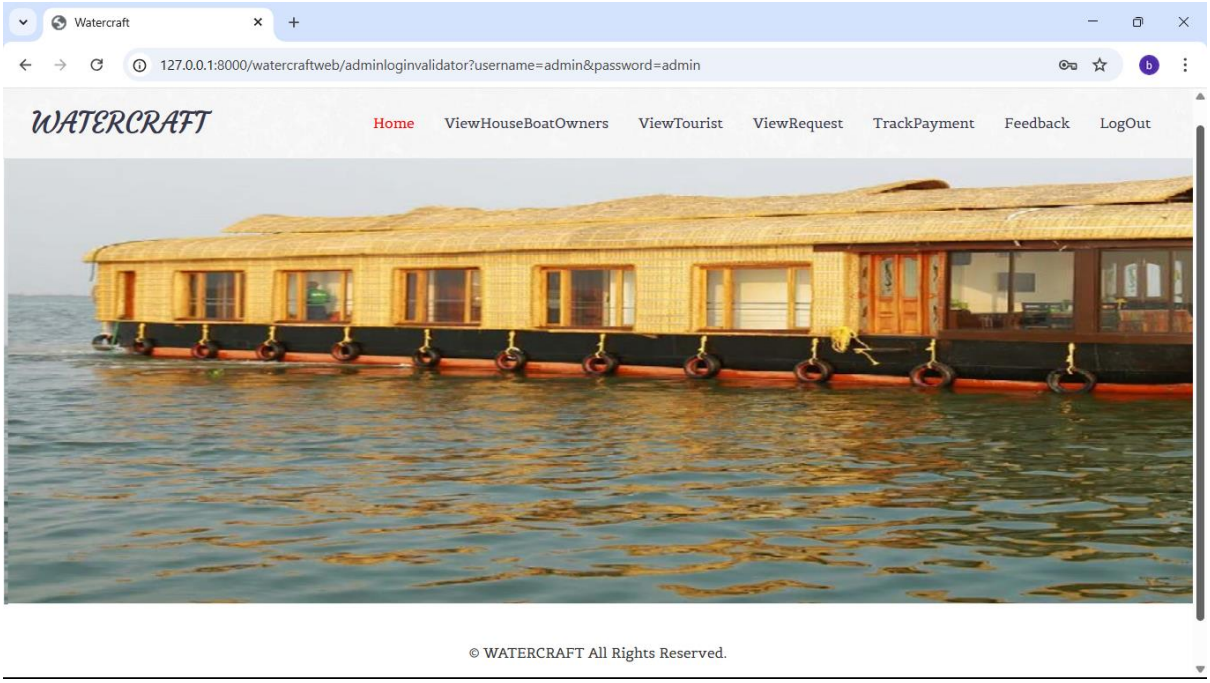


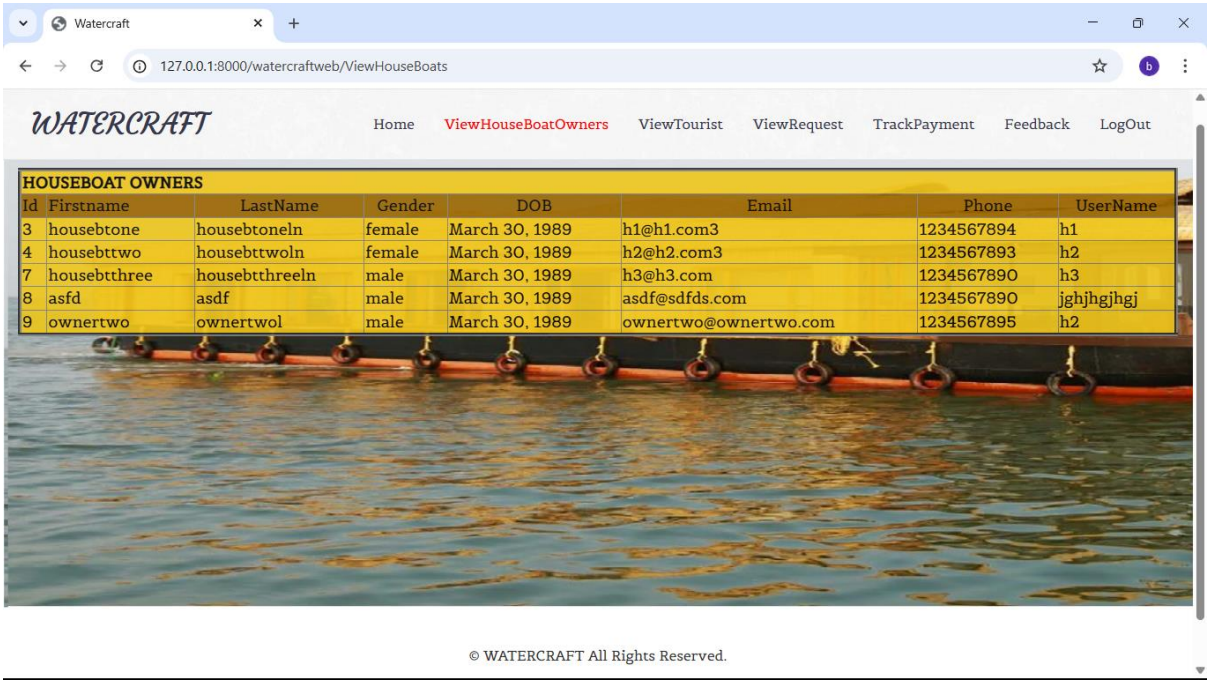




| houseboatid | boatname | boatdetails | boatimage | staydetails | pricedetails | uploaderid | uploadername | Date |
|---|---|---|---|---|---|---|---|---|
| 3 | asdf | asdfsdf | | asfdsdf | 3000 | 3 | h1 | 2024-01-01 |
| 5 | Boatthree | Boatthree Details | | Boatthree Stay details | 20000 | 3 | h1 | 2024-01-01 |

Admin

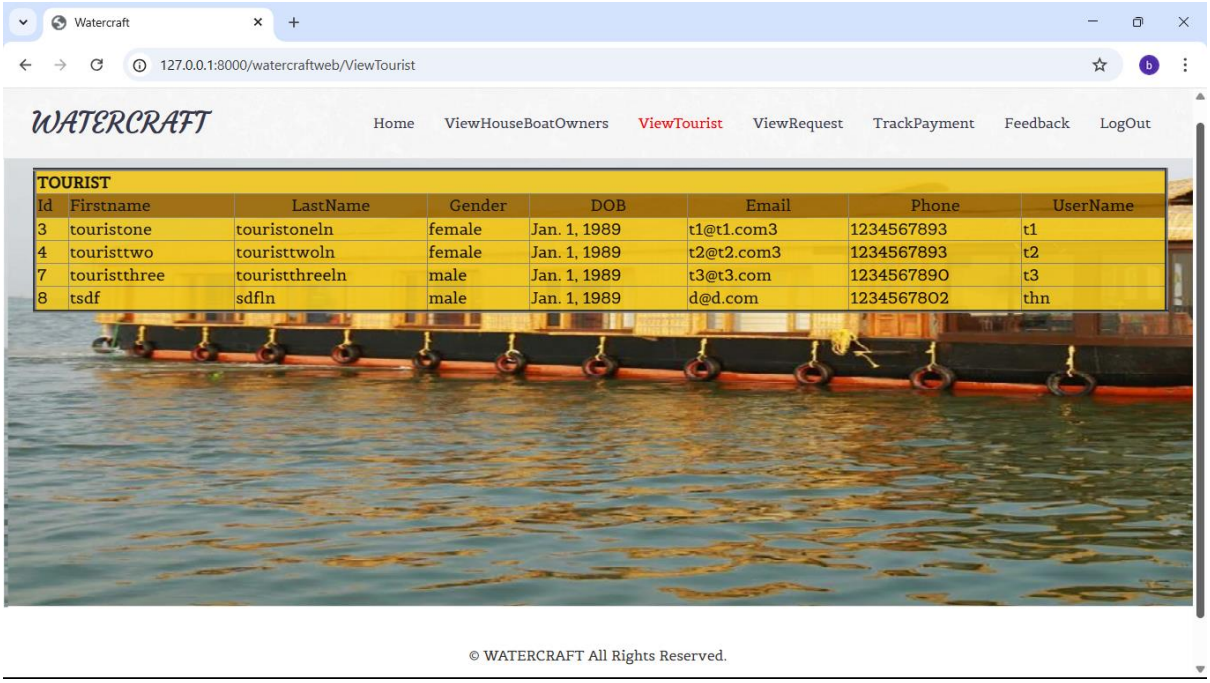

Watercraft
127.0.0.1:8000/watercraftweb/Adminlogin
WATERCRAFT
Home
ADMIN LOGIN
UserName
admin
Password
RESET
LOGIN
© WATERCRAFT All Rights Reserved.



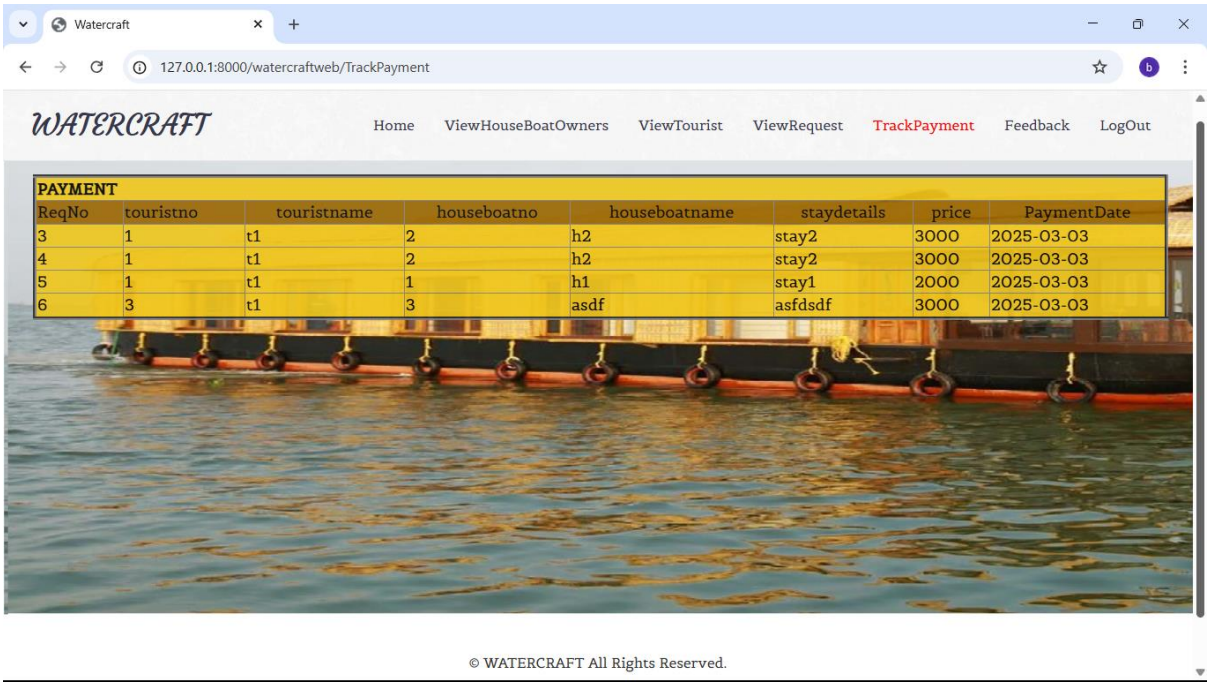

**HOUSEBOAT OWNERS**

| Id | Firstname | LastName | Gender | DOB | Email | Phone | UserName |
|---|---|---|---|---|---|---|---|
| 3 | housebtone | housebtoneln | female | March 30, 1989 | h1@h1.com3 | 1234567894 | h1 |
| 4 | housebttwo | housebttwoln | female | March 30, 1989 | h2@h2.com3 | 1234567893 | h2 |
| 7 | housebtthree | housebtthreeln | male | March 30, 1989 | h3@h3.com | 1234567890 | h3 |
| 8 | asfd | asdf | male | March 30, 1989 | asdf@sdfds.com | 1234567890 | jghjhgjhgj |
| 9 | ownertwo | ownertwol | male | March 30, 1989 | ownertwo@ownertwo.com | 1234567895 | h2 |




TOURIST

| Id | Firstname | LastName | Gender | DOB | Email | Phone | UserName |
|---|---|---|---|---|---|---|---|
| 3 | touristone | touristoneln | female | Jan. 1, 1989 | t1@t1.com3 | 1234567893 | t1 |
| 4 | touristtwo | touristtwoln | female | Jan. 1, 1989 | t2@t2.com3 | 1234567893 | t2 |
| 7 | touristthree | touristthreeln | male | Jan. 1, 1989 | t3@t3.com | 1234567890 | t3 |
| 8 | tsdf | sdfln | male | Jan. 1, 1989 | d@d.com | 1234567802 | thn |




PAYMENT

| ReqNo | touristno | touristname | houseboatno | houseboatname | staydetails | price | PaymentDate |
|---|---|---|---|---|---|---|---|
| 3 | 1 | t1 | 2 | h2 | stay2 | 3000 | 2025-03-03 |
| 4 | 1 | t1 | 2 | h2 | stay2 | 3000 | 2025-03-03 |
| 5 | 1 | t1 | 1 | h1 | stay1 | 2000 | 2025-03-03 |
| 6 | 3 | t1 | 3 | asdf | asfdsdf | 3000 | 2025-03-03 |

## D. Sample Coding

```
from django.shortcuts import render
import mysql.connector
from django.core.files.storage import FileSystemStorage
from datetime import datetime
import os

DB_CONFIG = {"host": "localhost", "user": "root", "password": "", "database":
"fluctuatingrenovatordb"}
BOAT_IMG_PATH = os.path.join("fluctuatingrenovatorweb", "static", "img", "boats")

def execute_query(query, values=None, fetch=True):
    try:
        with mysql.connector.connect(**DB_CONFIG) as mydb:
            with mydb.cursor() as mycursor:
                mycursor.execute(query, values)
                if fetch:
                    return mycursor.fetchall()
                else:
                    mydb.commit()
    except mysql.connector.Error as err:
        print(f"Database error: {err}")
        return None

def render_with_session(request, template_name, context=None):
    context = context or {}
    context['sname'] = request.session.get('sname')
    return render(request, template_name, context)

def index(request):
    return render(request, 'index.html')
```

```
def admin_login(request):
    return render(request, 'adminlogin.html')

def admin_home(request):
    return render_with_session(request, 'adminhome.html')

def admin_login_validator(request):
    u = request.GET.get("username")
    p = request.GET.get("password")
    if u == "admin" and p == "admin":
        request.session['sname'] = u
        return admin_home(request)
    return render(request, 'adminlogin.html', {'message': 'Invalid Credentials'})

def view_houseboats(request):
    rows = execute_query("SELECT * FROM houseboat")
    return render(request, 'ViewHouseBoats.html', {'rowf': rows})

def view_tourists(request):
    rows = execute_query("SELECT * FROM tourist")
    return render(request, 'ViewTourist.html', {'rowf': rows})

def view_requests(request):
    rows = execute_query("SELECT * FROM requesthandler")
    return render(request, 'ViewRequest.html', {'rowf': rows})

def track_payments(request):
    rows = execute_query("SELECT * FROM requesthandler WHERE status='paid'")
    return render(request, 'TrackPayment.html', {'rowf': rows})
```

```
def admin_view_feedback(request):
    rows = execute_query("SELECT * FROM feedback")
    return render(request, 'adminviewfeedback.html', {'rowf': rows})

def houseboat_login(request):
    return render(request, 'houseboatlogin.html')

def houseboat_login_validator(request):
    u = request.GET.get("username")
    p = request.GET.get("password")
    row = execute_query("SELECT * FROM houseboat WHERE username=%s AND
password=%s", (u, p), fetch=True)
    if row:
        request.session['snam'] = u
        request.session['sid'] = row[0][0]
        request.session['smob'] = row[0][6]
        request.session['semail'] = row[0][5]
        return render_with_session(request, 'houseboathome.html')
    return render(request, 'houseboatlogin.html', {'message': 'Invalid Credentials'})

def houseboat_home(request):
    return render_with_session(request, 'houseboathome.html')

def houseboat_register(request):
    return render(request, 'HouseboatRegister.html')

def houseboat_register_handler(request):
    data = {k: request.POST.get(k) for k in ["firstname", "lastname", "gender", "dob", "mobile",
"email", "username", "password"]}
    values = tuple(data.values())
    query = "INSERT INTO houseboat (firstname, lastname, gender, dob, phone, email,
```

```
username, password) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)"
    if execute_query(query, values, fetch=False):
        return render(request, 'HouseboatRegister.html', {'message': 'Successfully Added.'})
    return render(request, 'HouseboatRegister.html', {'message': 'Some issue. Try after some
time.'})

def handle_houseboat_profile(request):
    rows = execute_query("SELECT * FROM houseboat WHERE no=%s",
(request.session['sid'],))
    return render(request, 'HandleProfile.html', {'rowf': rows, 'sname':
request.session.get('snam')})

def houseboat_profile_edit_handler(request):
    data = {k: request.POST.get(k) for k in ["mobile", "email", "password"]}
    values = (data["mobile"], data["email"], data["password"], request.session['sid'])
    query = "UPDATE houseboat SET phone=%s, email=%s, password=%s WHERE no=%s"
    execute_query(query, values, fetch=False)
    return handle_houseboat_profile(request)

def add_houseboat_details(request):
    return render_with_session(request, 'AddHouseboatdetails.html')

def houseboat_add_houseboat(request):
    boatname = request.POST.get("boatname")
    bimage = None
    if request.method == 'POST' and 'boatimage' in request.FILES:
        file = request.FILES['boatimage']
        fs = FileSystemStorage(location=BOAT_IMG_PATH)
        filename = fs.save(boatname + file.name, file)
        bimage = filename
```

```
    data = {k: request.POST.get(k) for k in ["boatdetails", "staydetails", "pricedetails"]}
    uploaderid = request.session['sid']
    uploadername = request.session.get('snam')
    uploaddate = str(datetime.date(datetime.now()))
    values = (boatname, data["boatdetails"], bimage, data["staydetails"], data["pricedetails"],
uploaderid, uploadername, uploaddate)
    query = "INSERT INTO houseboatdetails (boatname, boatdetails, boatimage, staydetails,
pricedetails, uploaderid, uploadername, uploaddate) VALUES (%s, %s, %s, %s, %s, %s, %s,
%s)"
    if execute_query(query, values, fetch=False):
        return render(request, 'AddHouseboatdetails.html', {'sname': uploadername, 'message':
'Successfully Added.'})
    return render(request, 'AddHouseboatdetails.html', {'sname': uploadername, 'message': 'Some
issue. Try after some time.'})

def view_houseboat_details(request):
    rows = execute_query("SELECT * FROM houseboatdetails WHERE uploaderid=%s",
(request.session['sid'],))
    return render(request, 'ViewHouseboatdetails.html', {'rowf': rows, 'sname':
request.session.get('snam')})

def handle_tourist_request(request):
    rows = execute_query("SELECT * FROM requesthandler WHERE uploaderid=%s",
(request.session['sid'],))
    return render(request, 'HandleTouristRequest.html', {'rowf': rows, 'sname':
request.session.get('snam')})

def houseboat_request_handler(request):
    req_id = request.POST.get("selectedrequestid")
    execute_query("UPDATE requesthandler SET status='approved' WHERE reqno=%s",
(req_id,), fetch=False)
```

```
    return handle_tourist_request(request)

def view_payments(request):
    rows = execute_query("SELECT * FROM requesthandler WHERE status='paid' AND
uploaderid=%s", (request.session['sid'],))
    return render(request, 'ViewPayment.html', {'rowf': rows, 'sname':
request.session.get('snam')})

def tourist_login(request):
    return render(request, 'touristlogin.html')

def tourist_login_validator(request):
    u = request.GET.get("username")
    p = request.GET.get("password")
    row = execute_query("SELECT * FROM tourist WHERE username=%s AND
password=%s", (u, p), fetch=True)
    if row:
        request.session['snam'] = u
        request.session['sid'] = row[0][0]
        request.session['smob'] = row[0][6]
        request.session['semail'] = row[0][5]
        return render_with_session(request, 'touristhome.html')
    return render(request, 'touristlogin.html', {'message': 'Invalid Credentials'})

def tourist_home(request):
    return render_with_session(request, 'touristhome.html')

def tourist_register(request):
    return render(request, 'touristRegister.html')

def tourist_register_handler(request):
```

```
    data = {k: request.POST.get(k) for k in ["firstname", "lastname", "gender", "dob", "mobile",
"email", "username", "password"]}
    values = tuple(data.values())
    query = "INSERT INTO tourist (firstname, lastname, gender, dob, phone, email, username,
password) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)"
    if execute_query(query, values, fetch=False):
        return render(request, 'touristRegister.html', {'message': 'Successfully Added.'})
    return render(request, 'touristRegister.html', {'message': 'Some issue. Try after some time.'})

def view_tourist_profile(request):
    rows = execute_query("SELECT * FROM tourist WHERE no=%s", (request.session['sid'],))
    return render(request, 'ViewProfile.html', {'rowf': rows, 'sname': request.session.get('snam')})

def tourist_profile_edit_handler(request):
    data = {k: request.POST.get(k) for k in ["mobile", "email", "password"]}
    values = (data["mobile"], data["email"], data["password"], request.session['sid'])
    query = "UPDATE tourist SET phone=%s, email=%s, password=%s WHERE no=%s"
    execute_query(query, values, fetch=False)
    return view_tourist_profile(request)

def search_houseboats(request):
    rows = execute_query("SELECT * FROM houseboatdetails")
    return render(request, 'SearchHouseboats.html', {'rowf': rows, 'sname':
request.session.get('snam')})

def raise_request_handler(request):
    data = {k: request.POST.get(k) for k in ["houseboatno", "houseboatname", "staydetails",
"price", "uploaderid", "selecteddetails"]}
    touristno = request.session['sid']
    touristname = request.session.get('snam')
    date = str(datetime.date(datetime.now()))
```

```
    values = (touristno, touristname, data["houseboatno"], data["houseboatname"],
data["staydetails"], data["price"], date, data["uploaderid"], data["selecteddetails"])
    query = "INSERT INTO requesthandler (touristno, touristname, houseboatno, houseboatname,
staydetails, price, date, uploaderid, details) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)"
    if execute_query(query, values, fetch=False):
        return search_houseboats(request)
    return search_houseboats(request)

def handle_tourist_requests(request):
    rows = execute_query("SELECT * FROM requesthandler WHERE touristno=%s",
(request.session['sid'],))
    return render(request, 'HandleRequest.html', {'rowf': rows, 'sname':
request.session.get('snam')})

def tourist_pay_handler(request):
    req_id = request.POST.get("selectedrequestid")
    amount = request.POST.get("selectedamount")
    return render(request, 'touristpayhandler.html', {'dselectedrequestid': req_id,
'dselectedamount': amount, 'sname': request.session.get('snam')})

def tourist_pay_handler_updater(request):
    req_id = request.POST.get("selectedrequestid")
    execute_query("UPDATE requesthandler SET status='paid' WHERE reqno=%s", (req_id,),
fetch=False)
    return handle_tourist_requests(request)

def manage_payments(request):
    rows = execute_query("SELECT * FROM requesthandler WHERE status='paid' AND
touristno=%s", (request.session['sid'],))
    return render(request, 'ManagePayment.html', {'rowf': rows, 'sname':
request.session.get('snam')})
```

```
def tourist_post_feedback(request):
    return render_with_session(request, 'tpostfeedback.html')

def tourist_feedback_post_handler(request):
    date = str(datetime.date(datetime.now()))
    raisedid = request.session['sid']
    raisedname = request.session.get('snam')
    raisedtype = 'tourist'
    feedbackname = request.POST.get("feedback")
    feedbackdetails = request.POST.get("fdetails")
    values = (raisedid, raisedname, raisedtype, feedbackname, feedbackdetails, date)
    query = "INSERT INTO feedback (raisedid, raisedname, raisedtype, feedbackname,
feedbackdetails, date) VALUES (%s, %s, %s, %s, %s, %s)"
    if execute_query(query, values, fetch=False):
        return render(request, 'tpostfeedback.html', {'sname': raisedname, 'message': 'Posted'})
    return render(request, 'tpostfeedback.html', {'sname': raisedname, 'message': 'Issue'})

def tourist_view_feedback(request):
    rows = execute_query("SELECT * FROM feedback WHERE raisedid=%s",
(request.session['sid'],))
    return render(request, 'tviewfeedback.html', {'rowf': rows, 'sname':
request.session.get('snam')})

##################################################
```